

第3章 贪心算法

(Greedy Algorithm)

主讲教师：刘春风

智能与计算学部

cfliu@tju.edu.cn

引论

□ 优化问题:贪心法常用于解优化问题。

□ 应用:

- (1) 货箱装船 (Container Loading) 问题
 - (2) 背包问题
 - (3) 哈夫曼编码问题
 - (4) 拓扑排序问题
 - (5) 最短路径问题
 - (6) 最小代价生成树
 - (7) 偶图覆盖问题
-

3.1 优化问题

□ 一个优化问题可以描述如下：

(1) **问题的解**为一复杂的结构，例如元组形式

$$(x_1, x_2, \dots, x_n) \quad x_i \in S_i$$

(2) **约束条件**: $B(x_1, \dots, x_n)$ (结构性的约束条件)

使 $B(x_1, \dots, x_n)$ 为true的元组称为可行解

(feasible solution) ;

(3) **目标函数** $f(x_1, \dots, x_n)$

□ 优化解即指使目标函数极大化(或极小化)的可行解，对应的目标函数值称为优化值。

例:货箱装船问题

- 设有 n 个集装箱, 集装箱大小一样, 第 i 个集装箱的重量为 w_i ($1 \leq i \leq n$), 设船的载重量为 c .
- 试设计一装船的方法使得装入的集装箱数目最多.

- 令 $(x_1, \dots, x_n)$ $x_i \in \{0, 1\}$ 代表一种装法

- 约束条件 $\sum_{i=1}^n w_i x_i \leq c$

- 目标函数 $\sum_{i=1}^n x_i$

- 极大化目标函数

3.2 贪心算法概览

- 很多优化问题是NP-难度问题，迄今找不到它们的多项式算法。所以计算上可行的方法就是求其近似解。
 - 贪心法是求近似解的主要途径。
 - 贪心算法总是作出在当前看来最好的选择。也就是说贪心算法并不从整体最优考虑，它所作出的选择只是在某种意义上的局部最优选择。当然，希望贪心算法得到的最终结果也是整体最优的。
 - 贪心算法不能对所有问题都得到整体最优解，但对许多问题它能产生整体最优解。如单源最短路径问题，最小生成树问题等。
 - 在一些情况下，即使贪心算法不能得到整体最优解，其最终结果却是最优解的很好近似。
-

贪心算法的基本要素

- 对于一个具体的问题，如何知道是否可用贪心算法解此问题，以及能否得到问题的最优解呢？这个问题很难给予肯定的回答。
 - 但是，从许多可以用贪心算法求解的问题中看到这类问题一般具有2个重要的性质：**贪心选择性质**和**最优子结构性质**。
-

贪心算法的基本要素

□ 贪心选择性质

- 所谓贪心选择性质是指所求问题的整体最优解可以通过一系列局部最优的选择，即贪心选择来达到。这是贪心算法可行的第一个基本要素，也是贪心算法与动态规划算法的主要区别。
 - 动态规划算法通常以自底向上的方式解各子问题，而贪心算法则通常以自顶向下的方式进行，以迭代的方式做出相继的贪心选择，每做一次贪心选择就将所求问题简化为规模更小的子问题。
 - 对于一个具体问题，要确定它是否具有贪心选择性质，必须证明每一步所做的贪心选择最终导致问题的整体最优解。
-

贪心算法的基本要素

□ 最优子结构性质

- 当一个问题最优解包含其子问题的最优解时，称此问题具有最优子结构性质。
 - 问题的最优子结构性质是该问题可用动态规划算法或贪心算法求解的关键特征。
-

3.3 贪心法的步骤

- ❑ 贪心法是一种多步(stage)求解的方法;
 - ❑ 每步按一种局部优化的策略选择解(元组)的一个分量;
 - ❑ 算法以第 n 步结束时构造出的对象(元组)作为问题的解.
 - ❑ 这种局部优化的策略又称为“贪心标准(greedy criterion).
-

贪心法的主要特点

- **不回溯**:选定一个分量后，不重试其它可能。
 - 使用**局部优化**策略的主要原因是减小计算开销。但局部优化策略不保证得到精确优化解，可能得到的是近似解，特别是对**NP**-难度问题。
 - **不同的“贪心”策略得到不同的算法**。
 - 常常采纳使目标函数有**最大增量**的策略为贪心策略，增量是局部性概念。
 - 遗传算法、神经网络等等都是具有这类贪心性质的启发式算法。
 - 贪心法常需要正确性证明
-

贪心法的正确性证明

数学归纳法

1. 叙述一个描述算法正确性的命题 $P(n)$, n 为算法步数或者问题规模
2. 归纳基础: $P(1)$ 或 $P(n_0)$ 为真, n_0 为某个自然数
3. 归纳步骤: $P(k) \Rightarrow P(k+1)$ 第一数学归纳法
 $\forall k(k < n) P(k) \Rightarrow P(n)$ 第二数学归纳法

交换论证

1. 分析算法的解的结构特征
2. 从一个最优解逐步进行结构变换(替换成分、交换次序等)
3. 证明上述变换最终得到算法的解、变换有限步结束、变换保持最优性不降低、

例：找零钱问题

- 假设有四种硬币，它们的面值分别为二角五分、一角、五分和一分。要找给某顾客六角三分钱。
- 这时，我们会不假思索地拿出**2**个二角五分的硬币，**1**个一角的硬币和**3**个一分的硬币交给顾客。
 - 这种找硬币方法与其他找法相比，所拿出的硬币个数是最少的。这里，我们下意识地使用了这样的找硬币算法：首先选出一个面值不超过六角三分的最大硬币，即二角五分；然后从六角三分中减去二角五分，剩下三角八分；再选出一个面值不超过三角八分的最大硬币，即又一个二角五分，如此一直做下去。这个找硬币的方法实际上就是贪心算法。
 - 顾名思义，贪心算法总是作出在当前看来是最好的选择。也就是说贪心算法并不从整体最优上加以考虑，它所作出的选择只是在某种意义上的局部最优选择。
-

例：活动安排问题

■ 活动安排问题

- 要在所给的活动集合中选出最大的相容活动子集合，
- 该问题要求高效地安排一系列争用某一公共资源的活动。
- 贪心算法提供了一个简单的方法使得尽可能多的活动能兼容地使用公共资源。

■ 设有 n 个活动的集合 $E=\{1,2,\dots,n\}$,

- 每个活动都要求使用同一资源，如演讲会场等，
 - 同一时间内只有一个活动能使用这一资源。
 - 每个活动 i 都有一个要求使用该资源的起始时间 s_i 和一个结束时间 f_i ,且 $s_i < f_i$ 。如果选择了活动 i ，则它在半开时间区间 $[s_i, f_i)$ 内占用资源。
 - 若区间 $[s_i, f_i)$ 与区间 $[s_j, f_j)$ 不相交，则称活动 i 与活动 j 是相容的。即当 $s_i \geq f_j$ 或 $s_j \geq f_i$ 时，活动 i 与活动 j 相容。
-

活动安排问题-贪心算法描述

- 下面所给出活动安排问题的贪心算法greedySelector :

```
public static int greedySelector(int [] s, int [] f, boolean a[])
{
    int n=s.length-1;
    a[1]=true;
    int j=1;
    int count=1;
    for (int i=2;i<=n;i++) {
        if (s[i]>=f[j]) {
            a[i]=true;
            j=i;
            count++;
        }
        else a[i]=false;
    }
    return count;
}
```

各活动的起始时间和
结束时间存储于数组s
和f中且按结束时间的
非减序排列

活动安排问题-贪心算法分析

- 由于输入的活动以其完成时间的**非减序排列**，所以算法greedySelector每次总是选择具有**最早完成时间**的相容活动加入集合A中。
 - 直观上，按这种方法选择相容活动为未安排活动留下尽可能多的时间。也就是说，该算法的贪心选择的意义是**使剩余的可安排时间段极大化**，以便安排尽可能多的相容活动。
 - 算法greedySelector的效率极高。当输入的活动已按结束时间的非减序排列，算法只需 **$O(n)$** 的时间安排n个活动，使最多的活动能相容地使用公共资源。
 - 如果所给出的活动未按非减序排列，可以用min堆重排。
 - 算法复杂度为 **$(n\log n)$**
-

算法正确性的证明

定理1 算法Select 执行到第 k 步, 选择 k 项活动 $i_1=1, i_2, \dots, i_k$, 那么存在最优解 A 包含 $i_1=1, i_2, \dots, i_k$

根据定理: 算法至多到第 n 步得到最优解

证: $S=\{1,2,\dots,n\}$ 是活动集, 且 $f_1 \leq \dots \leq f_n$

归纳基础: $k=1$, 证明存在最优解包含活动1

任取最优解 A , A 中的活动按截止时间递增排列. 如果 A 的第一个活动为 j , $j \neq 1$, 令

$$A' = (A - \{j\}) \cup \{1\},$$

由于 $f_1 \leq f_j$, A' 也是最优解, 且含有1.

算法正确性的证明（续）

归纳步骤：假设命题对 k 为真,证明对 $k+1$ 也为真.

算法执行到第 k 步, 选择了活动 $i_1=1, i_2, \dots, i_k$, 根据归纳假设存在最优解 A 包含 $i_1=1, i_2, \dots, i_k$,

A 中剩下的活动选自集合 $S'=\{i \mid i \in S, s_i \geq f_k\}$, 且

$$A = \{i_1, i_2, \dots, i_k\} \cup B$$

B 是 S' 的最优解. (若不然, S' 的最优解为 B^* , B^* 的活动比 B 多, 那么 $B^* \cup \{1, i_2, \dots, i_k\}$ 是 S 的最优解, 且比 A 的活动多, 与 A 的最优性矛盾.)

根据归纳基础, 存在 S' 的最优解 B' 含有 S' 中的第一个活动, 即 i_{k+1} , 且 $|B'|=|B|$, 于是

$$\{i_1, i_2, \dots, i_k\} \cup B' = \{i_1, i_2, \dots, i_k, i_{k+1}\} \cup (B' - \{i_{k+1}\})$$

也是原问题的最优解.

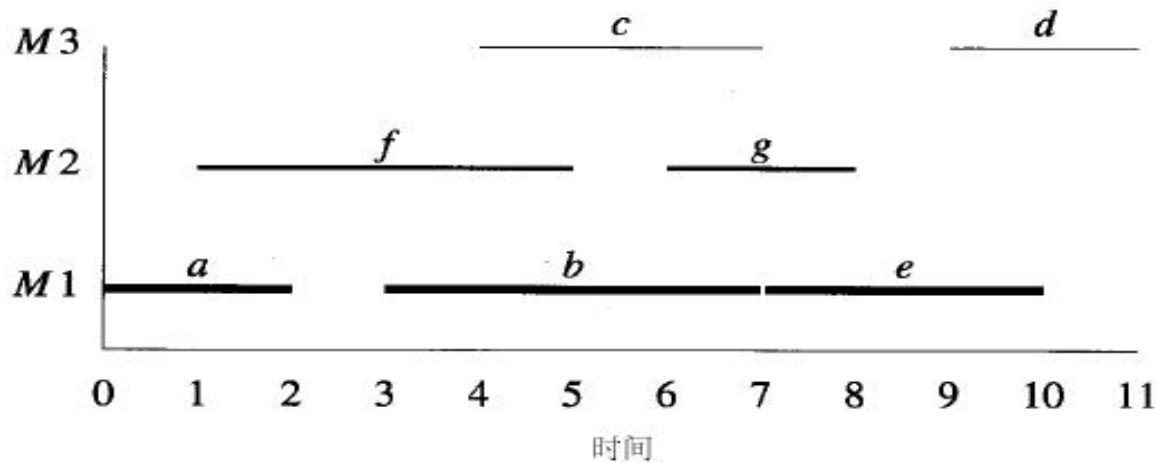
例：机器调度问题

- 现有 n 个任务和足够多台机器, 假定任何时间一台机器只能执行一个任务。设任务 i 的开始时间为 s_i , 完成时间为 f_i , $s_i < f_i$, $[s_i, f_i]$ 为处理任务 i 的时间区间。称两个任务 i, j 重叠是指两个任务的时间区间有重叠, 例如: 区间 $[1, 4]$ 与区间 $[2, 5]$ 重叠, 而与区间 $[4, 7]$ 不重叠, 求此机器调度问题的最优分配方案。
 - 提示: 可行的任务分配是指该分配中没有将重叠的任务分配给同一台机器。最优分配指占用机器数最少的可行分配。
-

机器调度问题

任务	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>
开始	0	3	4	9	7	1	6
完成	2	7	7	11	10	5	8

a)



b)

图3.1 7个任务使用三台机器的调度
a) 7个任务 b) 调度

机器调度问题-贪心算法设计

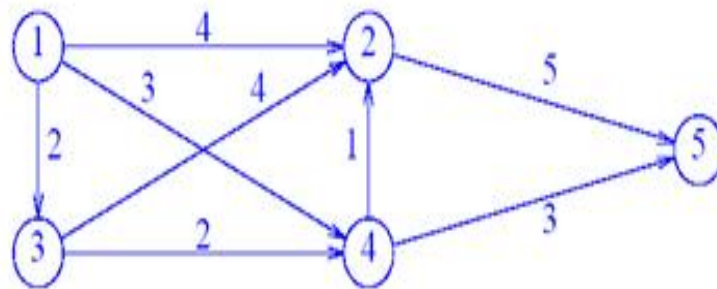
- 按任务起始时间对任务排序并按此顺序安排任务。
 - **贪心准则**：对当前要安排的任务尽可能使用旧(old)的机器。
 - 定义每台用过的机器的可用时间为这台机器上最近执行的任务的完成时间。
 - 如果一个新任务的起始时间 $\geq$ 这些机器的最小可用时间则安排该任务在这台机器上执行；否则使用一台新机器。
 - 使用min-堆存放每台机器的可用时间,即此时间以后可安排新任务.
 - 算法的时间复杂度为 $\Theta(n\log n)$ (排序和堆操作).
-

机器调度问题-贪心算法的性能

- 贪心算法并不总能求得问题的整体最优解。但对于活动安排问题、机器调度问题，上述贪心算法却总能求得整体最优解。
 - 证明：
 - 任何可行解使用的机器数 $\geq$ 最大重迭任务数；所以优化调度使用的机器数 $\geq$ 最大重迭任务数。
 - 贪心解使用的机器数不超过最大重迭任务数：任何时候当算法使用一台新机器时，当前这些机器上的任务一定是彼此重叠的。
 - 因此，使用贪心解所使用的机器数量等于最大重叠任务数，此即是最优解。
-

例：最短路径问题

- 给定一个有向图 G , 一个源节点 s , 目的节点 d , 找出从 s 到 d 的一条最短路径。
- 贪心算法：从 s 开始选择离 s “最近” 的下一个节点 q ; 再从 q 开始找和 q 相邻的且不在前面已选择的路径上的下一节点;
- 重复这一过程直到遇到 d 节点或该路径无法继续延伸。
- 例如：求该图中节点1到节点5的最短路径。
- 按贪心法找到的从1到5的路径为(1, 3, 4, 2, 5), 该贪心解不是优化解。



3.4 贪心算法的应用

- (1) 货箱装船 (Container Loading) 问题
 - (2) 背包问题 (knapsack problem)
 - (3) 拓扑排序问题
 - (4) 哈夫曼编码问题
 - (5) 最短路径问题
 - (6) 最小代价生成树
 - (7) 偶图覆盖问题
-

3.4.1 Container loading

- ❑ 目标函数：装载的集装箱数目。
 - ❑ 极大化目标函数。
 - ❑ 贪心标准：在还没装船的集装箱中选择重量最小的集装箱。
 - ❑ 按上述策略,重量最小的集装箱先装,依此类推。
 - ❑ 算法：首先按重量从小到大对集装箱排序并依次装入直到超过船的载重量。
-

定理3.1

- 上述贪心法产生的解是最优解。
 - 证明: 设货箱装船问题的最优解为 $(y_1, \dots, y_n)$ 。
 - 如最优解不含箱子1($y_1=0$), 将箱子1替换优化解中某一个箱子得到一个新的解。
 - 替换是必须的: 因为如果箱子1还能装入船中, 则 $(y_1, \dots, y_n)$ 不是优化的。
 - 因为箱子1是最轻的, 替换后的解仍是可行的。替换后的解装入的箱子数等于优化的装箱数, 所以它仍是优化解。
 - 替换后新的优化解和贪心解都包含箱子1。
 - 反复替换将得到一个优化解, 它等于贪心解。
 - 替换次数是有穷的。
-

3.4.2 0/1背包问题

□ 0/1背包问题：设有容量 c 的背包和 n 件物品，物品 i 的重量为 w_i ；假定装入物品 i 获得的效益值为 p_i ，试给出一装入物品的方法，使获得的总效益值最大。

□ 集装箱装载问题是0/1背包问题的特例：

$$p_i=1$$

□ 0/1背包问题是NP—难度问题，任何有多项式复杂度的算法产生的解都是近似解。

3.4.2 0/1背包问题

- 0/1背包问题可形式化描述如下:
 - 使用0/1数组 $(x_1, \dots, x_n)$ 表示一个装法:
 $x_i=1$ 表示装物品 i , 否则不装
 - 约束条件: $\sum_{i=1}^n w_i x_i \leq c$
 - 目标函数: $\sum_{i=1}^n p_i x_i$, 等于装入物品的总效益值
 - 极大化目标函数
-

0/1背包问题

□ 0/1背包问题有多种贪心策略

(1) 从未装入的物品中, 选出效益值最大的物品装入。利用这种规则, 效益值最大的物品首先被装入(假设有足够容量), 然后是下一个效益值最大的物品, 如此进行下去。

□ 例: $n=3, c=105, w=[100,10,10], p=[20,15,15]$

贪心解为: $[1,0,0]$, 效益值为: 20

优化解为: $[0,1,1]$, 效益值为30

□ 结论: 这种策略不一定能保证得到最优解

0/1背包问题-贪心策略

(2)从尚未装入的物品中选择重量最小的物品。虽然这一贪心法对于货箱装船问题能产生最优解,但在一般情况下不一定能得到最优解。

□ 例: $n=2, c=25, w=[10, 20], p=[5, 100]$

0/1背包问题-贪心策略

(3) 按密度 p_i/w_i ,从未装的物品中选择可装入背包(装入后不超过背包容量),且密度值最大的物品

例: $c=11, w=(2,4,6,5), p=(6,10,12,9)$

优化解为: $x=(1,1,0,1)$.

注: 这种策略也不能保证得到最优解

例: $n=3, c=30, w=[20,15,15], p=[40,25,25]$

贪心解为 $[1,0,0]$,但优化解为 $[0,1,1]$

0/1背包问题-贪心算法伪代码

□ 密度贪心法的伪代码:

(1)将物品按密度从大到小排序

(2)for ($i=1; i < n; i++$)

 if (物品 i 可装入到背包内) $x_i=1$ (装入)

 else $x_i=0$ (舍弃);

□ 算法的时间复杂度为 $O(n\log n)$

0/1背包问题的贪心算法改进

- 贪心法往往不能得到精确解，贪心解与最优解的误差用以下比值的百分比来度量：

$$| \text{优化值} - \text{贪心解值} | / \text{优化值} * 100\%$$

- 例： $n=2$, $c=y$, $w=[1, y]$, $p=[10, 9y]$

对所有 $y > 1$, 贪心解的效益值 = 10;

- 事实上，当 $y > 10/9$ 时, 优化效益值 = $9y$ 。
 - 不管优化值多大, 贪心解的值总是 10。
 - 误差为 $(9y - 10) / 9y = 1 - (10/9y)$ ，对足够大的 y , 误差可达到百分之百。
-

0/1背包问题：k-优化算法

- k-优化算法是上述密度贪心算法的改进，改进后其误差可控制在 $1/(k+1)*100\%$ 之内。

例如3-优化算法的误差为25%。

- k-优化算法也要先对物品按密度从大到小排序；
 - 先将一些物品装入背包，然后对其余物品用贪心法；
 - 预先装入的物品数不超过k。
 - 对所有物品数不超过k的物品子集执行上述过程，并从中找到有最大效益值的解作为k-优化算法的解。
 - 考虑如下0/1背包问题的2-优化算法
 $n=4, w=[2, 4, 6, 7], p=[6, 10, 12, 13], c=11$ 。
-

0/1背包问题：k-优化算法

- 当 $k=0$ 时, 同于前述的密度贪心法. 因此解为 $x=[1, 1, 0, 0]$, 效益值为16。
 - $k=1$ 时, 最初的子集为 $\{1\}, \{2\}, \{3\}, \{4\}$ 。
 - 子集 $\{1\}, \{2\}$ 产生与 $k=0$ 时相同的结果。
 - 考虑子集 $\{3\}$, 置 x_3 为1, 此时背包剩余容量为5, 未装的物品为1, 2, 4, 使用贪心法, 得到的解为 $x=[1, 0, 1, 0]$, 效益值为18。
 - 从子集 $\{4\}$ 开始, 产生的解为 $x=[1, 0, 0, 1]$, 效益值为19。
 - 因此, $k=0, 1$ 时获得的最优解为 $[1, 0, 0, 1]$, 效益值为19。
-

0/1背包问题：k-优化算法

- 若 $k=2$, 除了考虑 $k<2$ 的子集, 还必需考虑子集 $\{1,2\}$, $\{1,3\}$, $\{1,4\}$, $\{2,3\}$, $\{2,4\}$ 和 $\{3,4\}$.
 - 子集 $\{3,4\}$ 是不可行的, 将其舍弃
 - 对剩下的子集求解分别得到如下结果: $[1,1,0,0]$, $[1,0,1,0]$, $[1,0,0,1]$, $[0,1,1,0]$ 和 $[0,1,0,1]$, 最后一个的效益值为23。
 - 2-优化解为 $[0,1,0,1]$, 效益值为23
-

0/1背包问题：k-优化算法

- k-优化方法($k > 0$)得到的解误差不超过 $(1/(k+1)) * 100\%$
 - 当 $k=1$ 时,为50%以内,即如优化值为100,贪心法算出的值不低于50;
 - 当 $k=2$ 时,为33.33%以内。
 - 算法的时间复杂度随 k 的增大而增加:
 - 需要测试的子集数目为 $O(n^k)$;
 - 每一个子集做贪心法需时间 $O(n)$;
 - 因此当 $k > 0$ 时总的时间开销为 $O(n^{k+1})$ 。
-

0/1背包问题：k-优化算法

600种随机测试的统计结果

	偏差百分比				
k	0	1%	5%	10%	25%
0	239	390	528	583	600
1	360	527	598	600	
2	483	581	600		

600个例子中差值在x%以内的数目

3.4.3 哈夫曼编码问题

- 哈夫曼编码是广泛地用于数据文件压缩的十分有效的编码方法。其压缩率通常在20%~90%之间。哈夫曼编码算法用字符在文件中出现的频率表来建立一个用0, 1串表示各字符的最优表示方式。
- 给出现频率高的字符较短的编码, 出现频率较低的字符以较长的编码, 可以大大缩短总码长。
- 例如一个包含100,000个字符的文件, 各字符出现频率不同, 如下表所示。定长编码需要300,000位, 而按表中变长编码方案, 文件的总码长为:
 $(45 \times 1 + 13 \times 3 + 12 \times 3 + 16 \times 3 + 9 \times 4 + 5 \times 4) \times 1000 = 224,000$ 。

	a	b	c	d	e	f
频率(千次)	45	13	12	16	9	5
定长码	000	001	010	011	100	101
变长码	0	101	100	111	1101	1100

最优前缀码

二元前缀码

用0-1字符串作为代码表示字符，要求任何字符的代码都不能作为其它字符代码的前缀

非前缀码的例子

$a: 001, b: 00, c: 010, d: 01$

解码的歧义，例如字符串 0100001

解码1: 01, 00, 001 d, b, a

解码2: 010, 00, 01 c, b, d

- 使平均码长达到最小的前缀码编码方案称为给定编码字符集C的最优前缀码。

前缀码的二叉树及其权值

前缀码: { 00000, 00001, 0001, 001, 01, 100, 101, 11 }

频率: 00000: 5%, 00001: 5%, 0001: 10%, 001: 15%,
01: 25%, 100: 10%, 101: 10%, 11: 20%

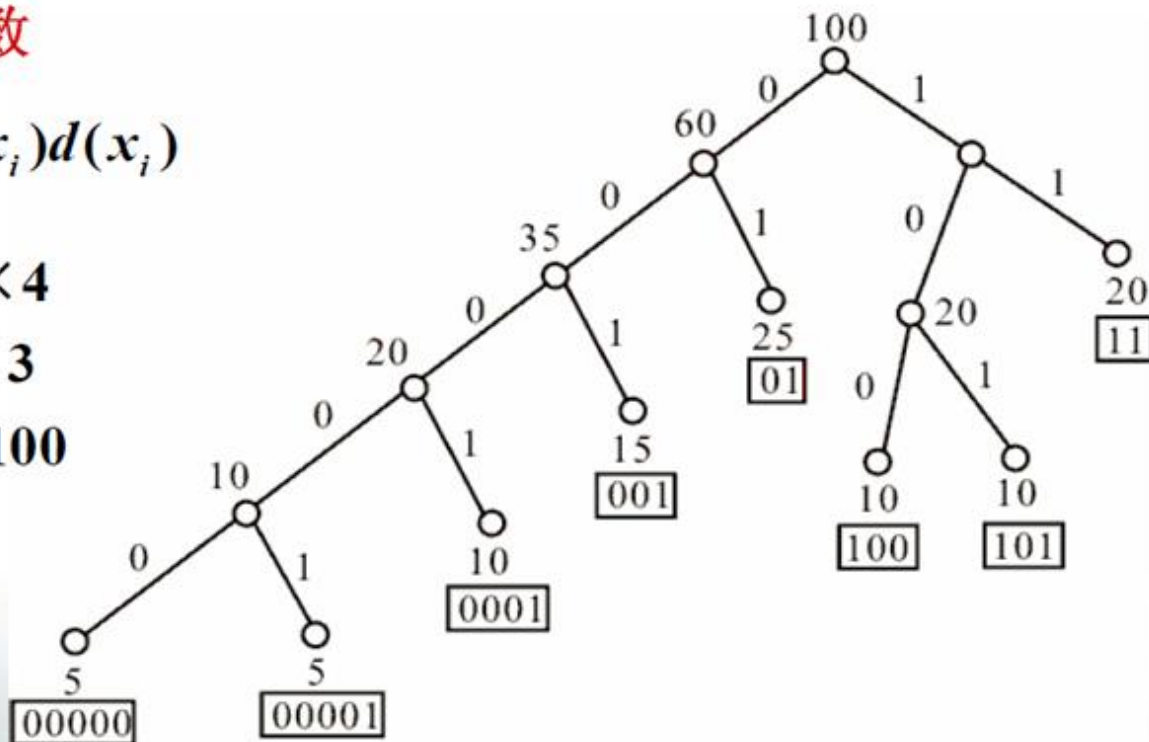
平均的二进制位数

$$B = \sum_{i=1}^n f(x_i) d(x_i)$$

$$\begin{aligned} B &= [(5+5) \times 5 + 10 \times 4 \\ &\quad + (15+10+10) \times 3 \\ &\quad + (25+20) \times 2] / 100 \\ &= 2.85 \end{aligned}$$

最优前缀码

权值 B 最小



最优前缀码问题

问题：给定字符集 $C=\{x_1, x_2, \dots, x_n\}$ 和每个字符的频率 $f(x)$ $i=1, 2, \dots, n$, 求关于 C 的一个最优前缀码.

算法 Huffman(C)

输入: $C=\{x_1, x_2, \dots, x_n\}$, $f(x_i)$, $i=1, 2, \dots, n$.

输出: Q // 队列

1. $n \leftarrow |C|$
2. $Q \leftarrow C$ // 按频率递增构成队列 Q
3. for $i \leftarrow 1$ to $n-1$ do
4. $z \leftarrow \text{Allocate-Node}()$ // 生成结点 z
5. $z.\text{left} \leftarrow Q$ 中最小元 // 取出 Q 最小元作 z 的左儿子
6. $z.\text{right} \leftarrow Q$ 中最小元 // 取出 Q 最小元作 z 的右儿子
7. $f(z) \leftarrow f(x) + f(y)$
8. $\text{Insert}(Q, z)$ // 将 z 插入 Q
9. Return Q

实例

例如 $a:45$, $b:13$; $c:12$;
 $d:16$; $e:9$; $f:5$

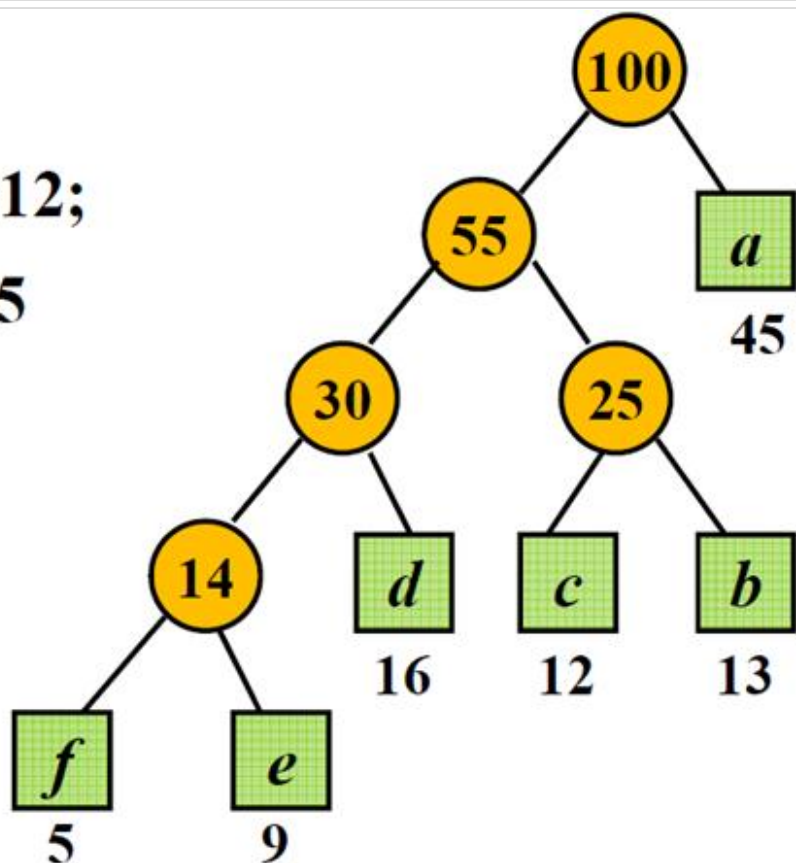
编码:

$f--0000$, $e--0001$,
 $d--001$, $c--010$,
 $b--011$, $a--1$

平均位数:

$4 \times (0.05 + 0.09)$

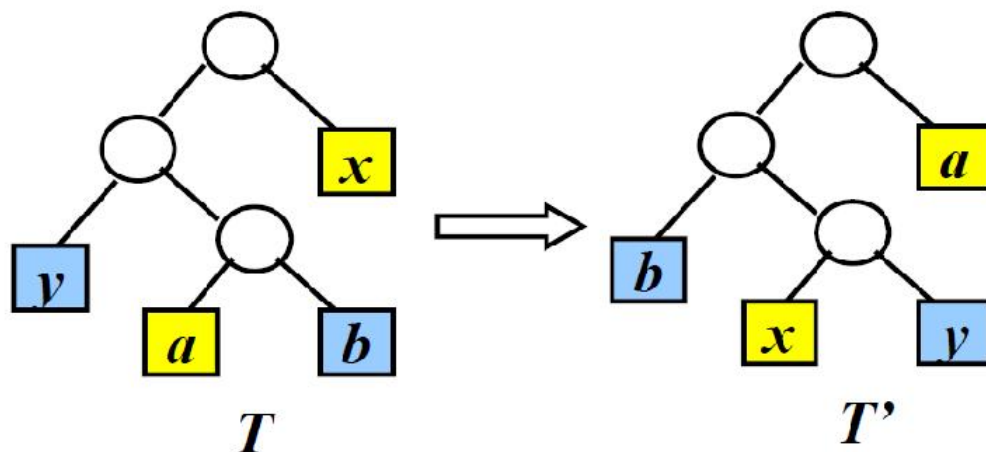
$+ 3 \times (0.16 + 0.12 + 0.13) + 1 \times 0.45 = 2.24$



正确性证明-涉及两个引理

引理1: 设 C 是字符集, $\forall c \in C, f(c)$ 为频率, $x, y \in C, f(x), f(y)$ 频率最小, 那么存在最优二元前缀码使得 x, y 的码字等长, 且仅在最后一位不同.

$T \rightarrow T'$
 $f[x] \leq f[a]$
 $f[y] \leq f[b]$
 a 与 x 交换
 b 与 y 交换



则 T 与 T' 的权之差为

$$B(T) - B(T') = \sum_{i \in C} f[i] d_T(i) - \sum_{i \in C} f[i] d_{T'}(i) \geq 0$$

其中 $d_T(i)$ 为 i 在 T 中的层数 (i 到根的距离)

正确性证明-涉及两个引理

引理 设 T 是二元前缀码所对应的二叉树, $\forall x, y \in T$, x, y 是树叶兄弟, z 是 x, y 的父亲, 令 $T' = T - \{x, y\}$, 且令 z 的频率 $f(z) = f(x) + f(y)$, T' 是对应于二元前缀码 $C' = (C - \{x, y\}) \cup \{z\}$ 的二叉树, 那么

$$B(T) = B(T') + f(x) + f(y).$$

证 $\forall c \in C - \{x, y\}$, 有 $d_T(c) = d_{T'}(c) \Rightarrow f(c)d_T(c) = f(c)d_{T'}(c)$

$$d_T(x) = d_T(y) = d_{T'}(z) + 1.$$

$$\begin{aligned} B(T) &= \sum_{i \in T} f(i)d_T(i) = \sum_{i \in T, i \neq x, y} f(i)d_T(i) + f(x)d_T(x) + f(y)d_T(y) \\ &= \sum_{i \in T', i \neq z} f(i)d_{T'}(i) + f(z)d_{T'}(z) + (f(x) + f(y)) \\ &= B(T') + f(x) + f(y) \end{aligned}$$

正确性证明：归纳法

定理 Huffman 算法对任意规模为 n ($n \geq 2$) 的字符集 C 都得到关于 C 的最优前缀码的二叉树.

归纳基础 $n=2$, 字符集 $C=\{x_1, x_2\}$, Huffman算法得到的代码是0和1, 是最优前缀码.

归纳步骤 假设Huffman算法对于规模为 k 的字符集都得到最优前缀码. 考虑规模为 $k+1$ 的字符集 $C=\{x_1, x_2, \dots, x_{k+1}\}$, 其中 $x_1, x_2 \in C$ 是频率最小的两个字符. 令

$$C' = (C - \{x_1, x_2\}) \cup \{z\}, \quad f(z) = f(x_1) + f(x_2)$$

根据归纳假设, Huffman算法得到一棵关于字符集 C' 、频率 $f(z)$ 和 $f(x_i)$ ($i=3, 4, \dots, k+1$) 的最优前缀码的二叉树 T' .

把 x_1 和 x_2 作为 z 的儿子附加到 T' 上, 得到树 T , 那么 T 是关于字符集 $C=(C' - \{z\}) \cup \{x_1, x_2\}$ 的最优前缀码的二叉树.

算法分析

- 统计编码字符集中每一字符 c 的频率 $f(c)$ 。以 f 为键值的优先队列 Q 用在贪心选择时有效地确定算法当前要合并的2棵具有最小频率的树。一旦2棵具有最小频率的树合并后，产生一棵新的树，其频率为合并的2棵树的频率之和，并将新树插入优先队列 Q 。经过 $n-1$ 次的合并后，优先队列中只剩下一棵树，即所要求的树 T 。
 - 算法可以用最小堆实现优先队列 Q 。初始化优先队列需要 $O(n)$ 计算时间，由于最小堆的插入和删除运算均需 $O(\log n)$ 时间， $n-1$ 次的合并总共需要 $O(n \log n)$ 计算时间。因此，关于 n 个字符的哈夫曼算法的计算时间为 $O(n \log n)$ 。
-

拓展：哈夫曼树的应用

问题：给定一组不同长度的排好序文件构成的集合

$$S = \{f_1, \dots, f_n\}$$

其中 f_i 表示第 i 个文件含有的项数. 使用二分归并将这些文件归并成一个有序的文件.

归并过程对应于二叉树：文件为树叶. f_i 与 f_j 归并的文件是它们的父结点.

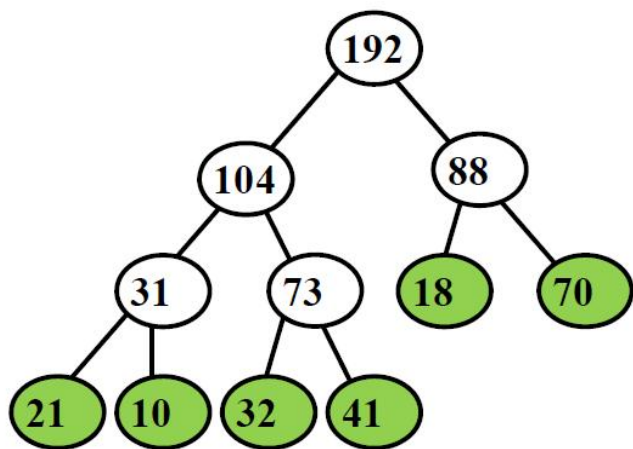
归并代价(最多的比较次数)：结点 f_i 与 f_j 归并代价为 $f_i + f_j - 1$.

总的代价：每个文件(树叶)的深度乘以文件大小之和再减掉归并次数 $n-1$

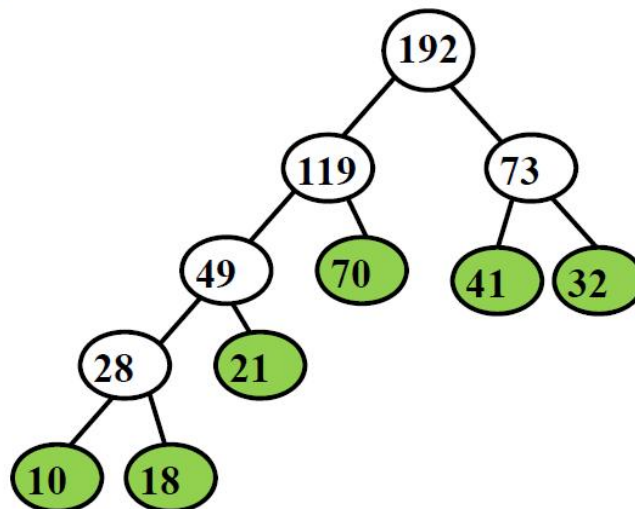
$$\sum_{i \in S} d(i) f_i - (n - 1)$$

哈夫曼树的应用

实例: $S = \{ 21, 10, 32, 41, 18, 70 \}$



顺序归并



Huffman树归并

代价

顺序归并: $(21+10+32+41) \times 3 + (18+70) \times 2 - 5 = 483$

Huffman树归并: $(10+18) \times 4 + 21 \times 3 + (70+41+32) \times 2 - 5 = 456$

3.4.4 单源最短路径问题

- 任给一有向图 G ,它的每条边都有一个非负的权值 $a[i][j]$, 路径的长度定义为路径上边的权值之和。
 - 单源最短路问题:给定源节点 s ,找出从 s 到图中所有其他节点(称为目的)的最短路径(优化解)及其长度(优化值)。
 - 例:
 - 网络中传输数据流时,要耗费网络的带宽和存储资源,使用跳数少的路径节省网络资源。
 - IP路由使用最短路算法(OSPF),因为IP路由表按目的节点索引(查找),所以,OSPF协议使用该算法求出网络中目的节点到任一节点的最短路。
-

Dijkstra's 最短路算法

- 如果链路权值非负, 则最短路的子路径也是最短路, 其长度小于原来路径的长度, 所以, 长度较小的最短路容易找到。
 - 贪心策略: 按最短路长度从小到大依次求解。
 - Dijkstra's 最短路算法使用上述贪心策略, 是图论算法中应用最为广泛的算法, 主要原因是其计算复杂度低且容易实现。
-

Dijkstra's 最短路算法基本步骤

- 维护一个集合S，该集合中源节点到其他节点的最短路已知，初始时该集合为空；
 - 从V-S集合中找一节点v，满足源节点到该节点距离最小；
 - 更新v的邻节点的到源节点的距离值。
-

Dijkstra's 算法伪代码

$d[s] \leftarrow 0$

for each $v \in V - \{s\}$

do $d[v] \leftarrow \infty$

$S \leftarrow \emptyset$

$Q \leftarrow V$ $\triangleleft Q$ is a priority queue maintaining

while $Q \neq \emptyset$

do $u \leftarrow \text{EXTRACT-MIN}(Q)$

$S \leftarrow S \cup \{u\}$

for each $v \in \text{Adj}[u]$

do if $d[v] > d[u] + w(u, v)$

then $d[v] \leftarrow d[u] + w(u, v)$

Dijkstra's 算法伪代码

$d[s] \leftarrow 0$

for each $v \in V - \{s\}$

do $d[v] \leftarrow \infty$

$S \leftarrow \emptyset$

$Q \leftarrow V$ $\triangleleft Q$ is a priority queue maintaining $V - S$

while $Q \neq \emptyset$

do $u \leftarrow \text{EXTRACT-MIN}(Q)$

$S \leftarrow S \cup \{u\}$

for each $v \in \text{Adj}[u]$

do if $d[v] > d[u] + w(u, v)$

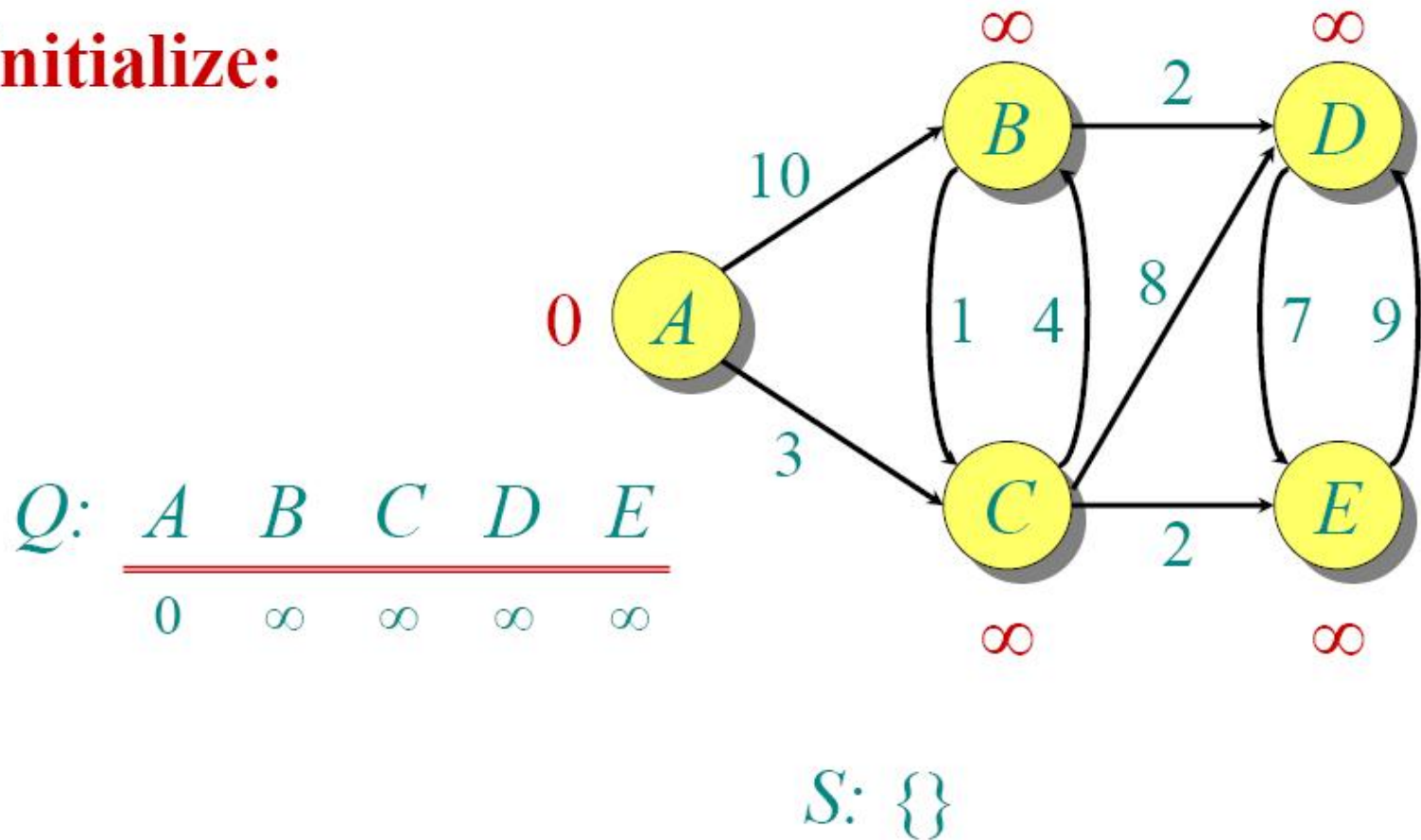
then $d[v] \leftarrow d[u] + w(u, v)$

*relaxation
step*

Implicit DECREASE-KEY

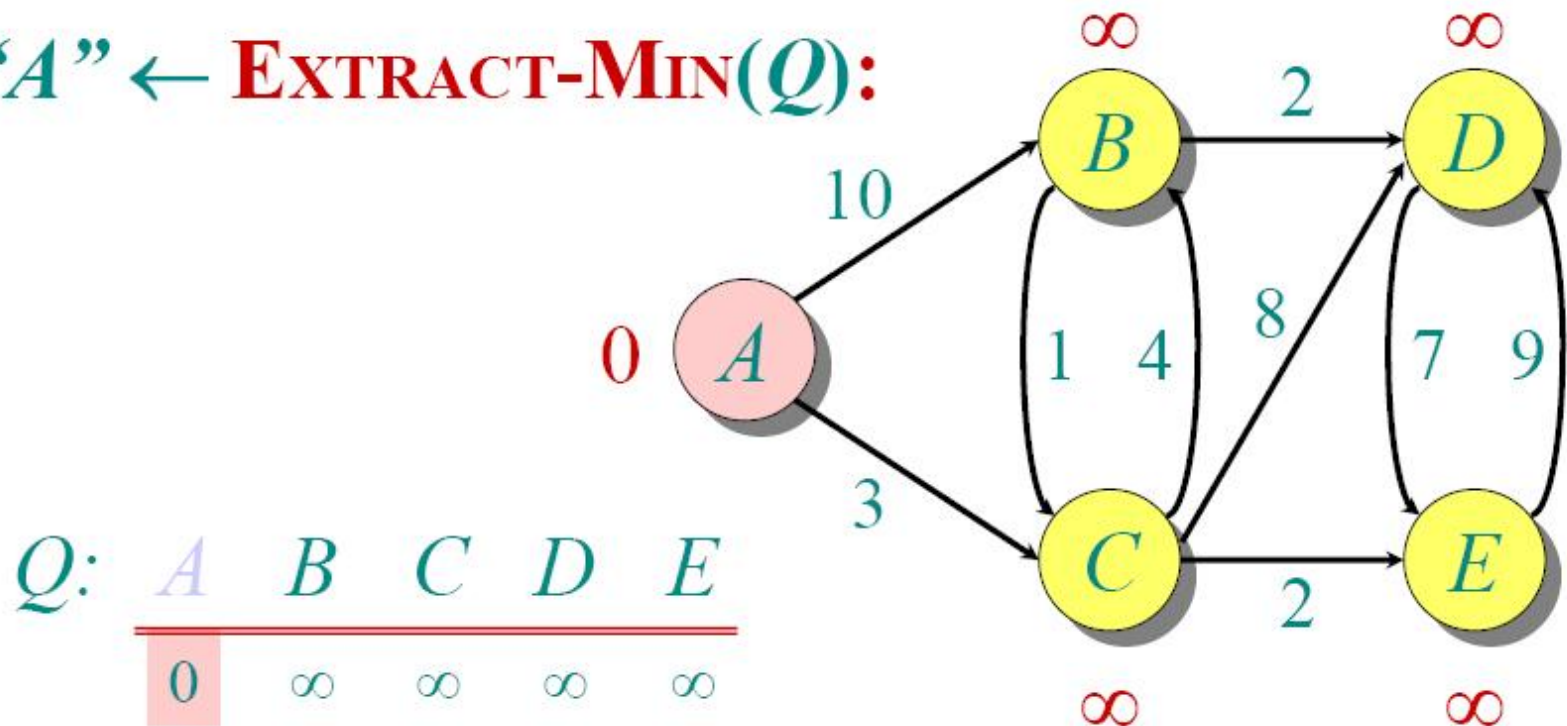
Dijkstra's 算法-实例

Initialize:



Dijkstra's 算法-实例

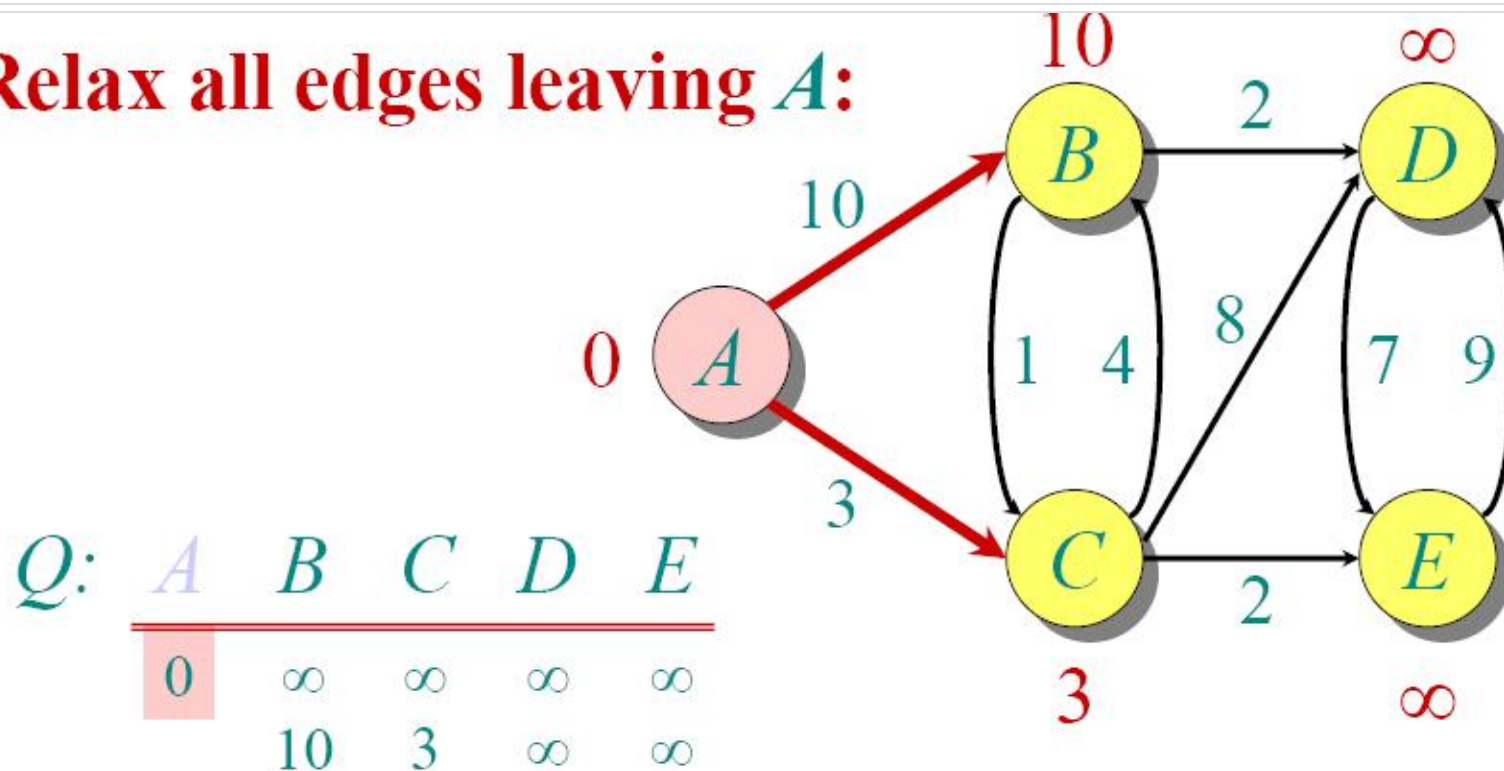
“A” $\leftarrow$ **EXTRACT-MIN**(Q):



$S: \{ A \}$

Dijkstra's 算法-实例

Relax all edges leaving A :



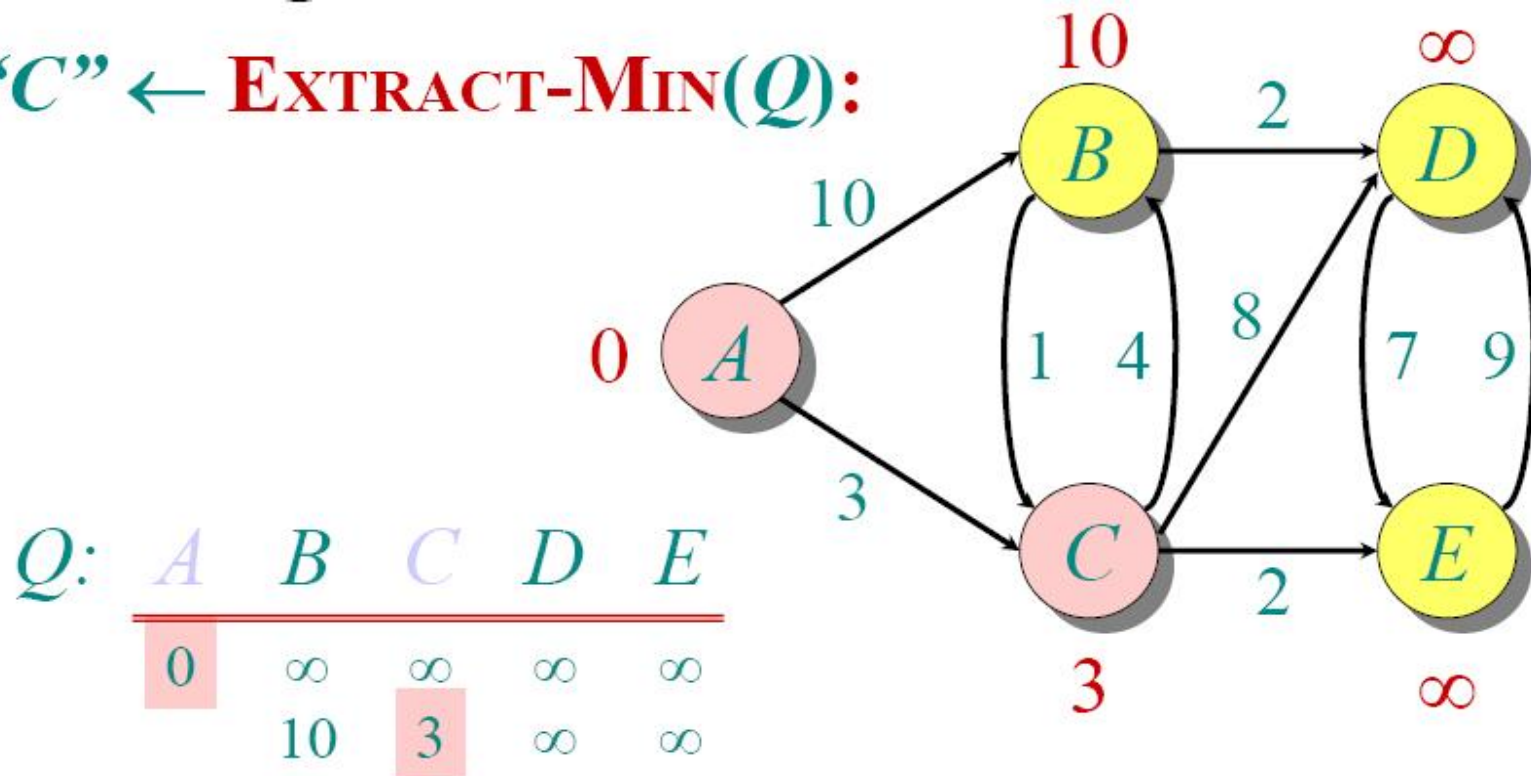
Q :

A	B	C	D	E
0	∞	∞	∞	∞
	10	3	∞	∞

$S: \{A\}$

Dijkstra's 算法-实例

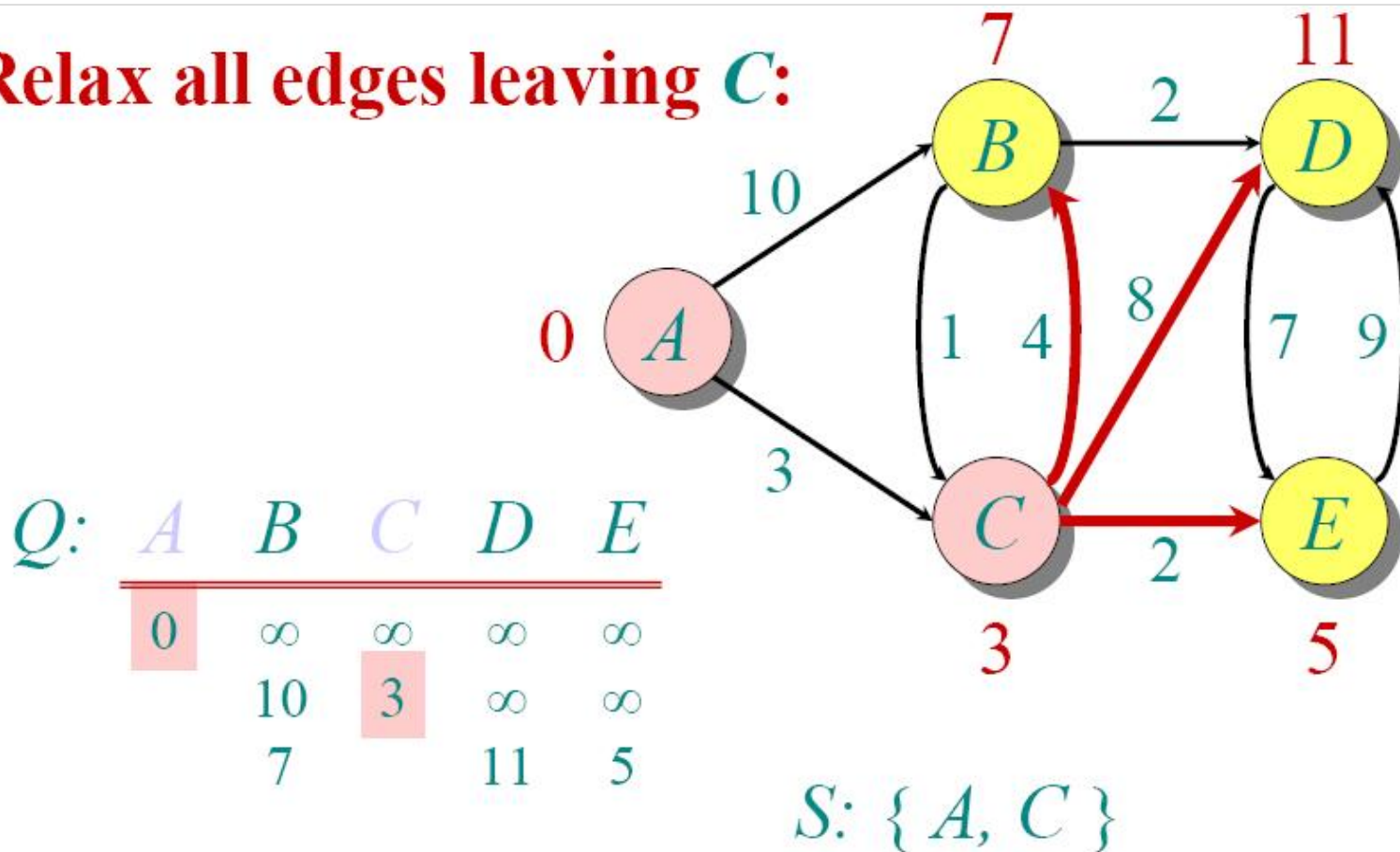
“C” $\leftarrow$ **EXTRACT-MIN**(Q):



$S: \{A, C\}$

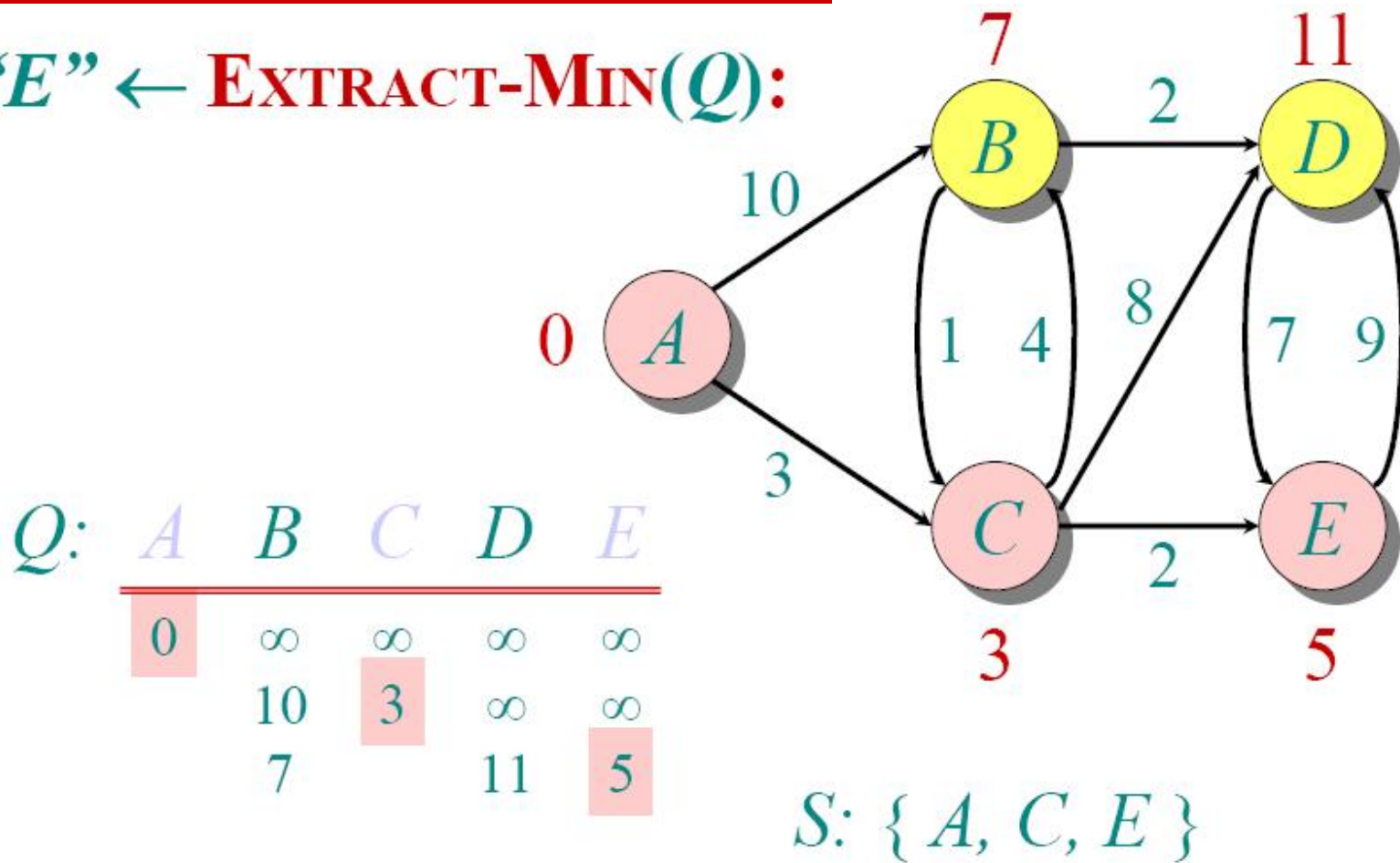
Dijkstra's 算法-实例

Relax all edges leaving **C**:



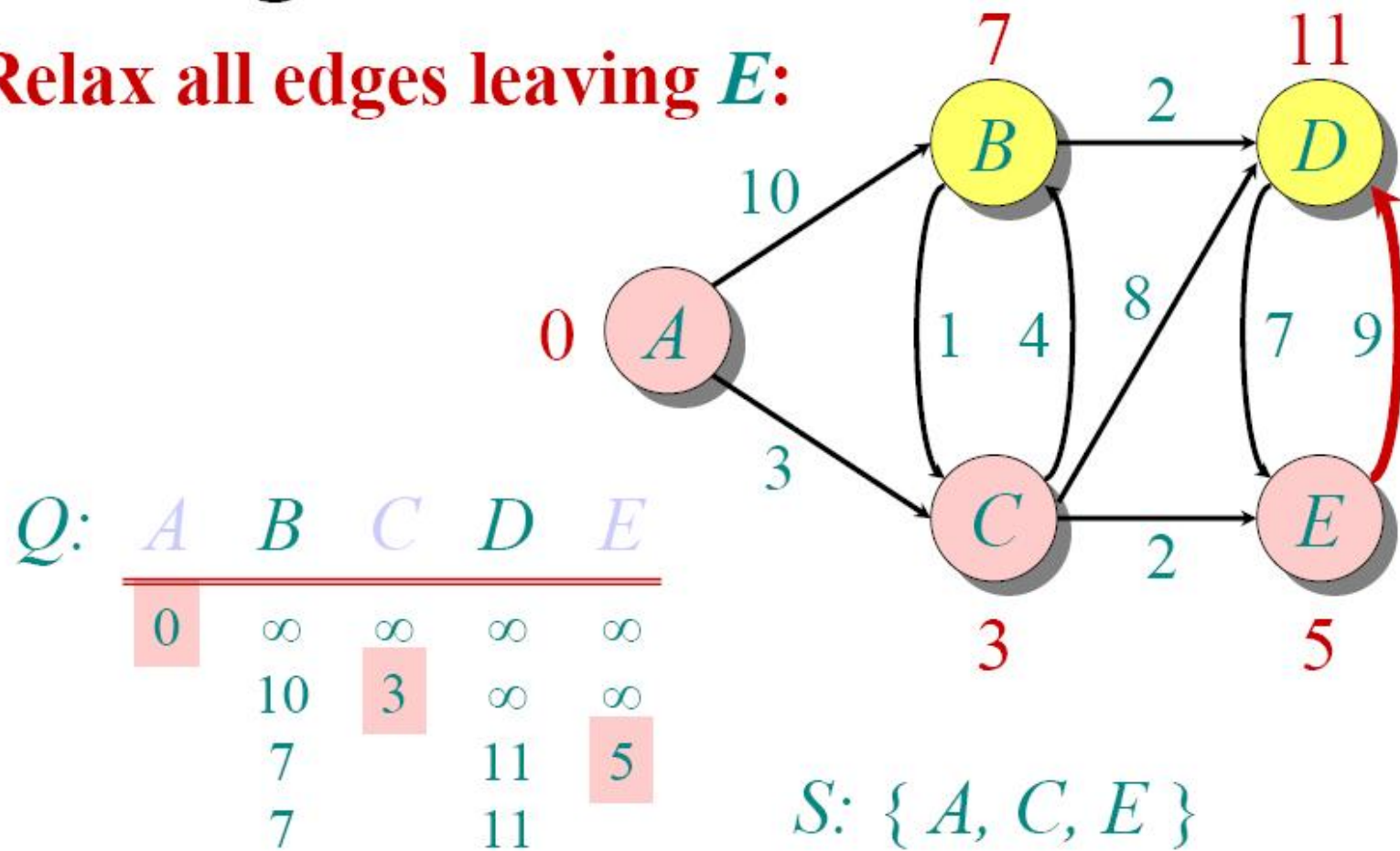
Dijkstra's 算法-实例

"E" ← EXTRACT-MIN(Q):



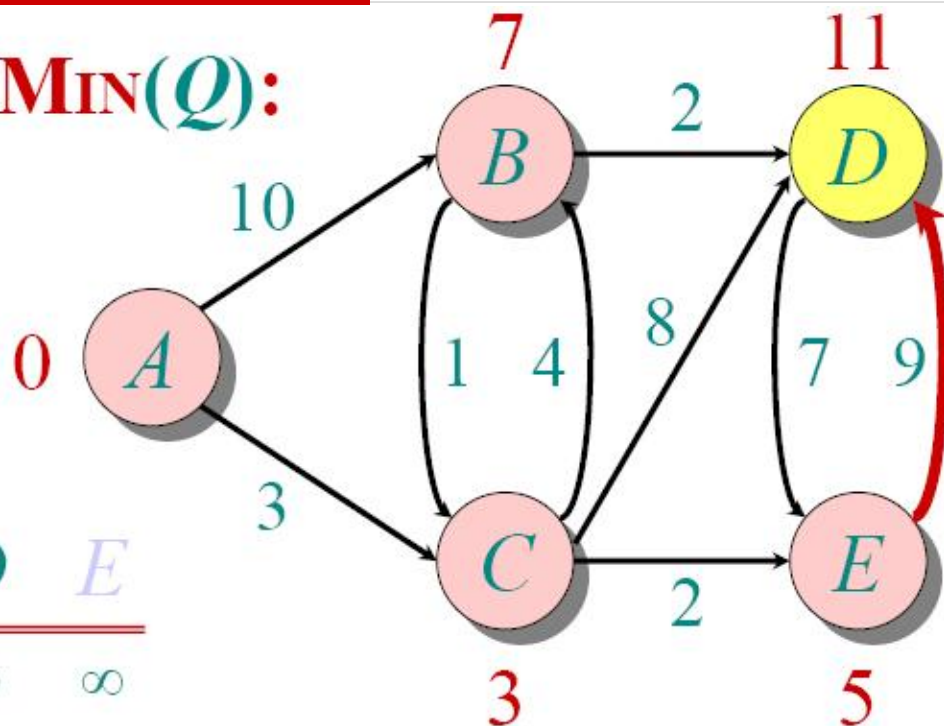
Dijkstra's 算法-实例

Relax all edges leaving *E*:



Dijkstra's 算法-实例

"B" ← EXTRACT-MIN(Q):



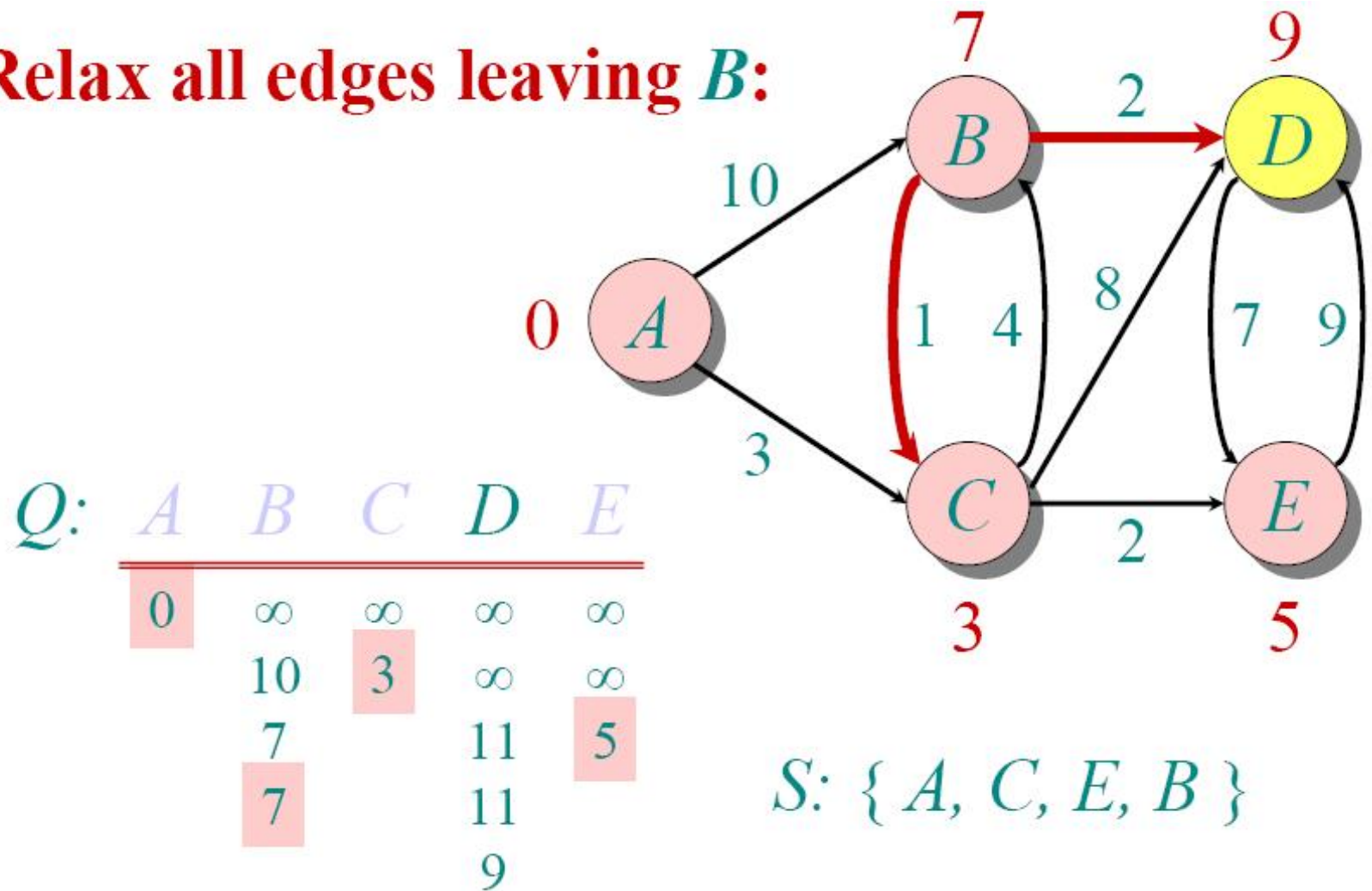
Q:

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
0	∞	∞	∞	∞
	10	3	∞	∞
	7		11	5
	7		11	

S: { *A*, *C*, *E*, *B* }

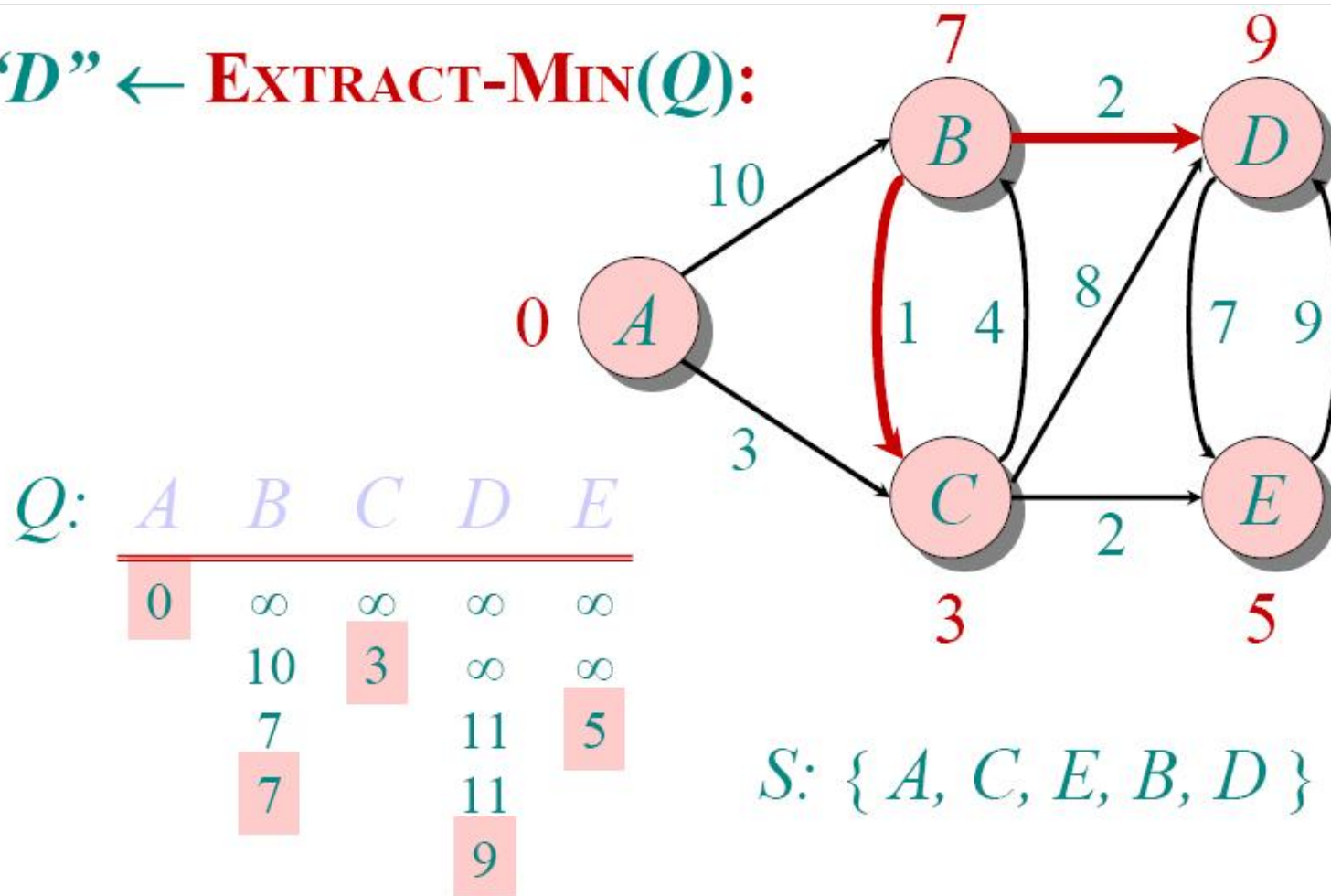
Dijkstra's 算法-实例

Relax all edges leaving *B*:



Dijkstra's 算法-实例

"D" ← EXTRACT-MIN(Q):



Dijkstra's 算法分析

```
while  $Q \neq \emptyset$   
  do  $u \leftarrow \text{EXTRACT-MIN}(Q)$   
     $S \leftarrow S \cup \{u\}$   
    for each  $v \in \text{Adj}[u]$   
      do if  $d[v] > d[u] + w(u, v)$   
        then  $d[v] \leftarrow d[u] + w(u, v)$ 
```

Dijkstra's 算法分析

$|V|$ times { while $Q \neq \emptyset$
do $u \leftarrow \text{EXTRACT-MIN}(Q)$
 $S \leftarrow S \cup \{u\}$
for each $v \in \text{Adj}[u]$
do if $d[v] > d[u] + w(u, v)$
then $d[v] \leftarrow d[u] + w(u, v)$

$\text{degree}(u)$ times {

Handshaking Lemma $\Rightarrow \Theta(E)$ implicit DECREASE-KEY's.

$$\text{Time} = \Theta(V) \cdot T_{\text{EXTRACT-MIN}} + \Theta(E) \cdot T_{\text{DECREASE-KEY}}$$

Dijkstra's 算法分析

$$\text{Time} = \underbrace{\Theta(V)}_Q \cdot \underbrace{T_{\text{EXTRACT-MIN}}}_{T_{\text{EXTRACT-MIN}}} + \underbrace{\Theta(E)}_{T_{\text{DECREASE-KEY}}} \cdot \underbrace{T_{\text{DECREASE-KEY}}}_{T_{\text{DECREASE-KEY}}}$$

Q	$T_{\text{EXTRACT-MIN}}$	$T_{\text{DECREASE-KEY}}$	Total
array	$O(V)$	$O(1)$	$O(V^2)$
binary heap	$O(\lg V)$	$O(\lg V)$	$O(E \lg V)$
Fibonacci heap	$O(\lg V)$ amortized	$O(1)$ amortized	$O(E + V \lg V)$ worst case

每个Extract-min操作的平摊代价

Dijkstra's 算法正确性证明

命题： 当算法进行到第 k 步时，对于 S 中每个结点 i ,

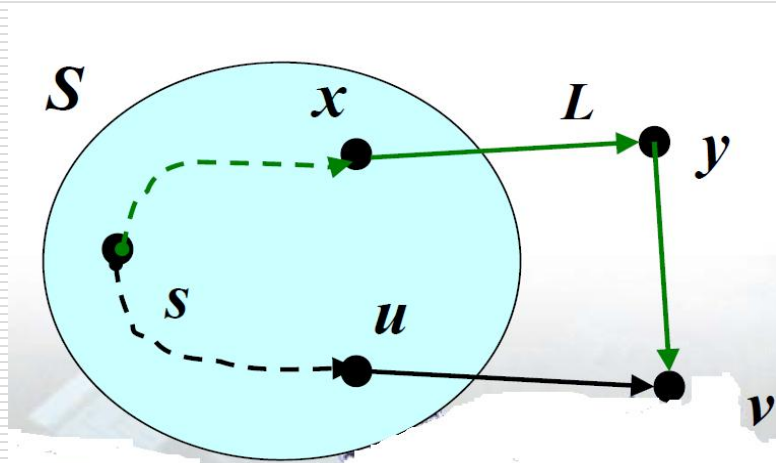
$$\text{dist}[i] = \text{short}[i]$$

归纳基础 $k=1, S=\{s\}, \text{dist}[s]=\text{short}[s]=0$, 命题为真.

归纳步骤 假设命题对于 k 为真. 考虑 $k+1$ 步, 选择顶点 v (边 $\{u, v\}$). 假若存在另一条 s - v 路径 L (绿色), 最后一次出 S 的顶点为 x , 在这次从 S 中出来后经过 $V-S$ 的第一个顶点为 y .

$$\begin{aligned} \text{dist}[v] &\leq \text{dist}[y] && //v \text{ 先被选} \\ &\leq \text{dist}[y] + d(y, v) \leq L \end{aligned}$$

$$\text{dist}[v] = \text{short}[v]$$



无权图 (Unweighted graphs)

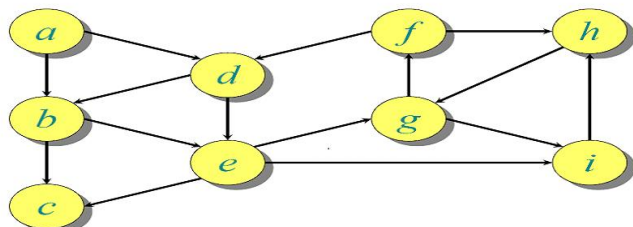
- 假定, 对所有点对 $(u,v) \in E$, $w(u,v)=1$, Dijkstra 算法能否进一步改进?
- 可使用FIFO队列代替优先队列
- 广度优先算法如下:

- *Breadth-first search*

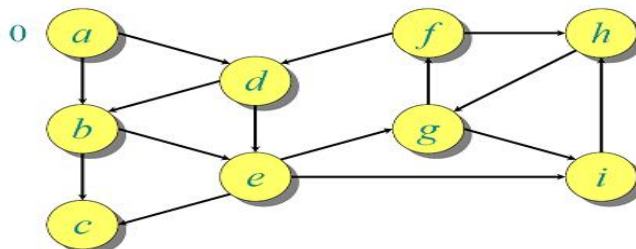
```
while  $Q \neq \emptyset$ 
do  $u \leftarrow \text{DEQUEUE}(Q)$ 
  for each  $v \in \text{Adj}[u]$ 
    do if  $d[v] = \infty$ 
      then  $d[v] \leftarrow d[u] + 1$ 
          ENQUEUE( $Q, v$ )
```

Analysis: Time = $O(V + E)$.

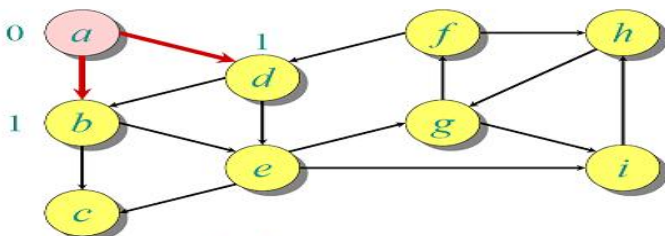
无权图 (Unweighted graphs)



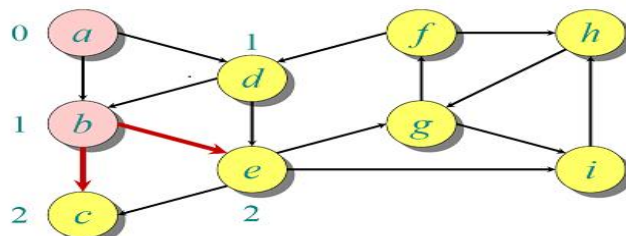
$Q:$



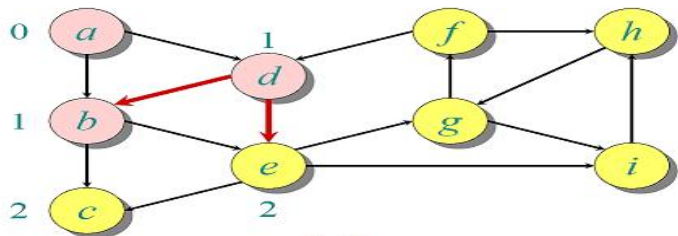
$Q: a$



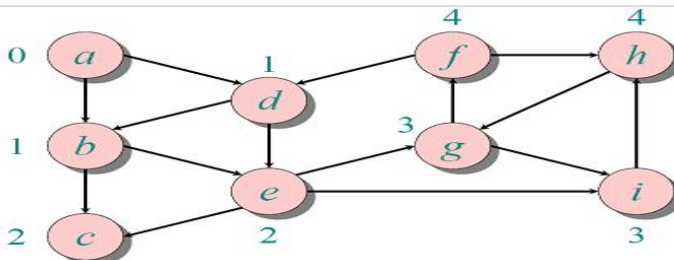
$Q: a \ b \ d$



$Q: a \ b \ d \ c \ e$



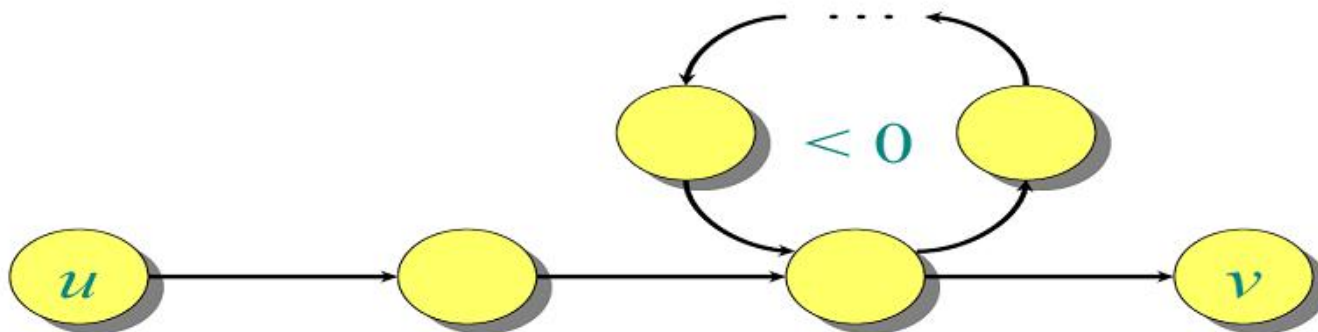
$Q: a \ b \ d \ c \ e$



$Q: a \ b \ d \ c \ e \ g \ i \ f \ h$

负权图(Negative-weight cycles)

- ❑ 若图 $G=(V,E)$ 包含负权回路，则最短路径可能不存在。
- ❑ 例



- ❑ **Bellman-Ford** 算法：找出源节点到其它任意节点的最短路，或确定图中包含负值环路
-

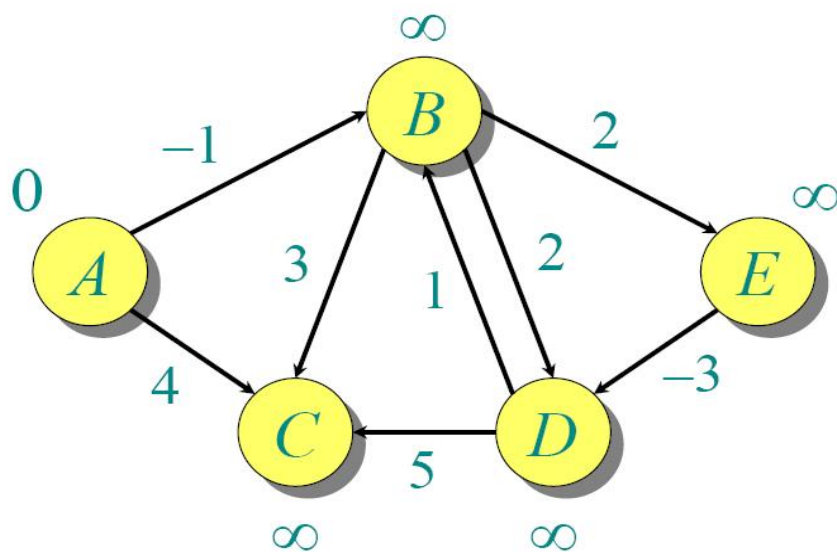
Bellman-Ford 算法

Bellman-Ford algorithm

```
 $d[s] \leftarrow 0$   
for each  $v \in V - \{s\}$   
  do  $d[v] \leftarrow \infty$  } initialization  
  
for  $i \leftarrow 1$  to  $|V| - 1$   
  do for each edge  $(u, v) \in E$   
    do if  $d[v] > d[u] + w(u, v)$   
      then  $d[v] \leftarrow d[u] + w(u, v)$  } relaxation step  
  
for each edge  $(u, v) \in E$   
  do if  $d[v] > d[u] + w(u, v)$   
    then report that a negative-weight cycle exists  
  
At the end,  $d[v] = \delta(s, v)$ . Time =  $O(VE)$ .
```

Bellman-Ford 算法-实例

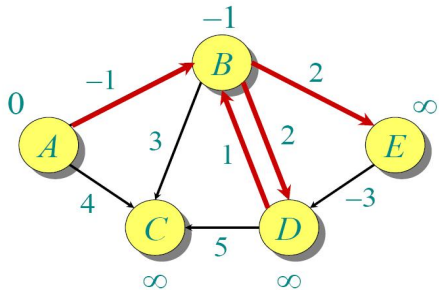
Example of Bellman-Ford



<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
0	∞	∞	∞	∞

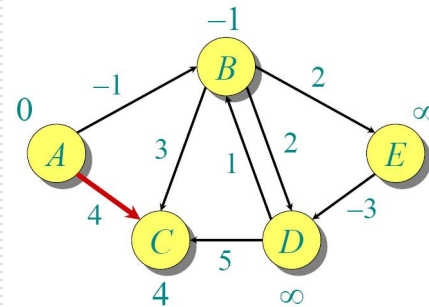
Bellman-Ford 算法-实例

Example of Bellman-Ford



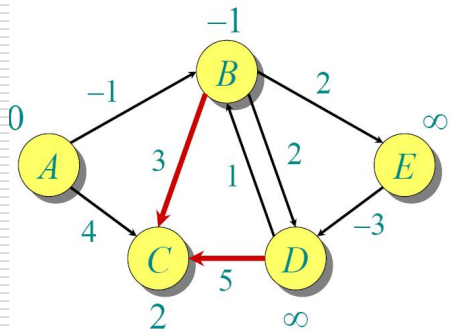
A	B	C	D	E
0	∞	∞	∞	∞
0	-1	∞	∞	∞

Example of Bellman-Ford



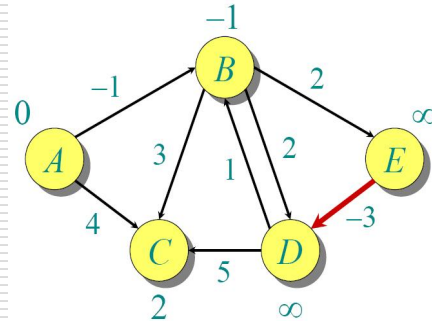
A	B	C	D	E
0	∞	∞	∞	∞
0	-1	∞	∞	∞
0	-1	4	∞	∞

Example of Bellman-Ford



A	B	C	D	E
0	∞	∞	∞	∞
0	-1	∞	∞	∞
0	-1	4	∞	∞
0	-1	2	∞	∞

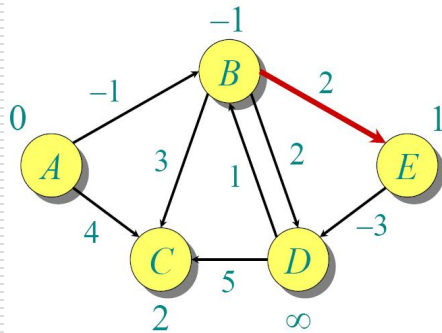
Example of Bellman-Ford



A	B	C	D	E
0	∞	∞	∞	∞
0	-1	∞	∞	∞
0	-1	4	∞	∞
0	-1	2	∞	∞

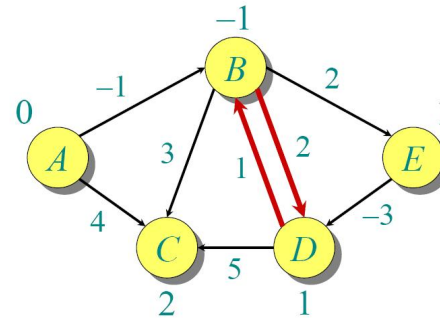
Bellman-Ford 算法-实例

Example of Bellman-Ford



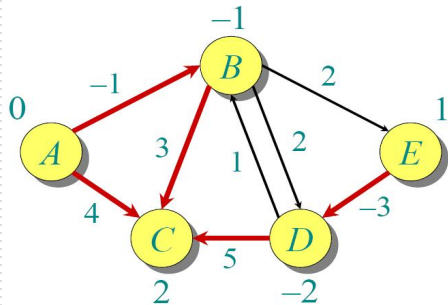
A	B	C	D	E
0	∞	∞	∞	∞
0	-1	∞	∞	∞
0	-1	4	∞	∞
0	-1	2	∞	∞
0	-1	2	∞	1

Example of Bellman-Ford



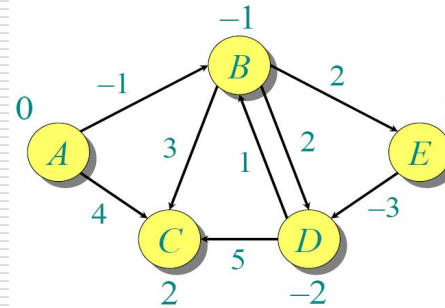
A	B	C	D	E
0	∞	∞	∞	∞
0	-1	∞	∞	∞
0	-1	4	∞	∞
0	-1	2	∞	∞
0	-1	2	∞	1
0	-1	2	1	1

Example of Bellman-Ford



A	B	C	D	E
0	∞	∞	∞	∞
0	-1	∞	∞	∞
0	-1	4	∞	∞
0	-1	2	∞	∞
0	-1	2	∞	1
0	-1	2	1	1
0	-1	2	-2	1

Example of Bellman-Ford



A	B	C	D	E
0	∞	∞	∞	∞
0	-1	∞	∞	∞
0	-1	4	∞	∞
0	-1	2	∞	∞
0	-1	2	∞	1
0	-1	2	1	1
0	-1	2	-2	1

Note: Values decrease monotonically.

最短路径问题小结

□ 单源最短路径

■ 非负权重

□ Dijkstra 算法复杂度: $O(E + V \lg V)$

■ 一般情况下:

□ Bellman-Ford 算法复杂度 $O(VE)$

□ 多源最短路径

■ 非负权值,

□ Dijkstra 算法复杂度 $O(VE + V^2 \lg V)$

■ 一般情况 (权值可能为负)

□ Bellman-Ford 算法为 $O(V^2E)$ 或 $O(V^4)$

3.4.5 单源最短路径问题

- 具有 n 个顶点的连通的无向图 G , 图的每条边 e 有一非负权值 $c(e)$, 也称为成本, 求有最小成本的生成树.
 - 每个生成树刚好具有 $n-1$ 条边, 所以问题是用某种方法选择 $n-1$ 条边使它们形成 G 的最小生成树.
 - 我们采用以下二种不同的贪心策略来选择这 $n-1$ 条边。
不同贪心策略对应求解最小生成树的二个算法：
Kruskal's算法, **Prim's算法**.
-

Kruskal's 算法

输入：连通图 G // 顶点数 n ，边数 m

输出： G 的最小生成树

1. 按权从小到大排序 G 中的边，使得 $E=\{e_1, e_2, \dots, e_m\}$

2. $T \leftarrow \emptyset$

3. repeat

4. $e \leftarrow E$ 中的最短边

5. if e 的两端点不在同一个连通分支

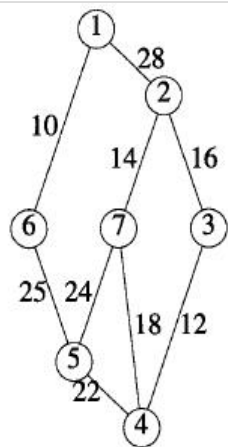
6. then $T \leftarrow T \cup \{e\}$

7. $E \leftarrow E - \{e\}$

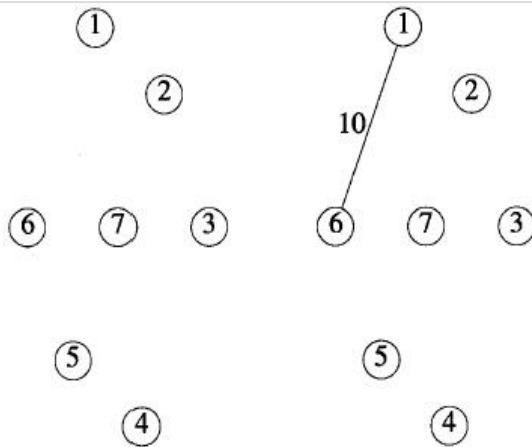
8. until T 包含了 $n-1$ 条边

贪心策略：每次选择权值 $c(e)$ 最小且与以前选择的边不构成 cycle 的边 e .

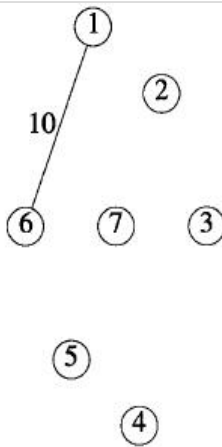
Kruskal's 算法 - 实例



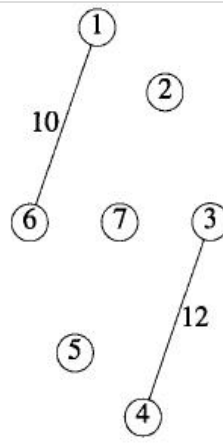
a)



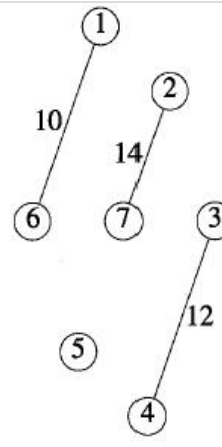
b)



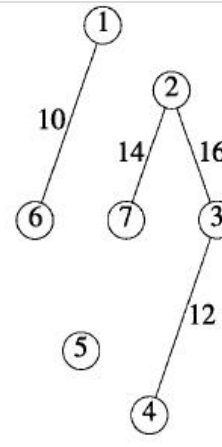
c)



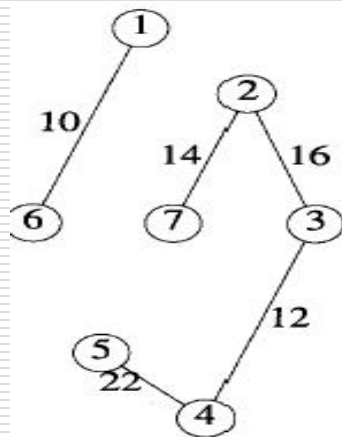
d)



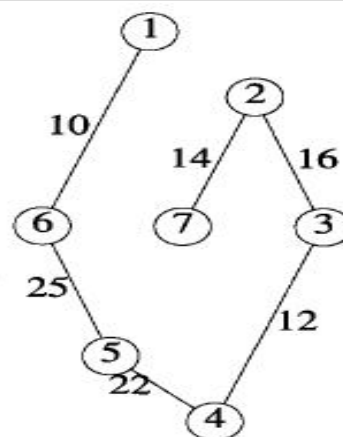
e)



f)



g)



h)

Kruskal's算法正确性证明

命题：对于任意 $n>1$, 算法对 n 阶图得到一棵最小生成树.

证明 $n=2$, 只有一条边, 命题显然为真.

假设对于 n 个顶点的图算法正确, 考虑 $n+1$ 个顶点的图 G , G 中最小权边 $e = \{i, j\}$, 从 G 中短接 i 和 j , 得到图 G' . 根据归纳假设, 由算法存在 G' 的最小生成树 T' . 令 $T = T' \cup \{e\}$, 则 T 是关于 G 的最小生成树.

否则存在 G 的含边 e 的最小生成树 T^* , $W(T^*) < W(T)$. (如果 $e \notin T^*$, 在 T^* 中加边 e , 形成回路. 去掉回路中任意别的边所得生成树的权仍旧最小). 在 T^* 中短接 e 得到 G' 的生成树 $T^* - \{e\}$, 且

$$W(T^* - \{e\}) = W(T^*) - w(e) < W(T) - w(e) = W(T'),$$
与 T' 的最优性矛盾.

Kruskal's算法实现—数据结构

- ❑ 算法执行过程中动态产生的子树, 反复进行union和find操作, 使用下面的数据结构很有效.
- ❑ UNION—FIND数据结构
- ❑ 初始为单个顶点的集合
- ❑ 对每条边做2次FIND找到边的端点所在的集合; 如果在同一集合中则舍弃该条边, 否则将2个集合合并(UNION)
- ❑ 算法可在 $O(n+e\log e)$ 找出最小生成树:

边排序: $O(e\log e)$

initializing: $O(n)$

union-find: $O(e)$

Prim's算法

- 设A为算法执行过程中产生的生成树中的节点集合，初始化A包含图中任一节点;
 - 定义V-A中节点u到A的距离 $\text{key}(u)$ 为
$$\min\{c(u,v) | v \in A, (u,v) \in E\}$$
 - 取V-A中 $\text{key}(u)$ 最小的节点u, 并将其加入到A;
 - 更新V-A中的节点到A的距离(key值);
 - 重复执行上述步骤。
-

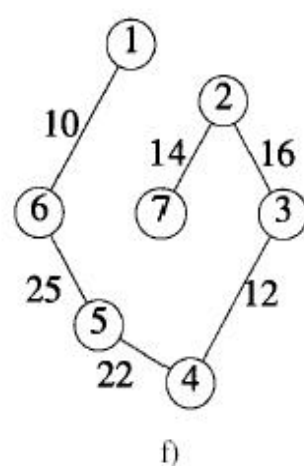
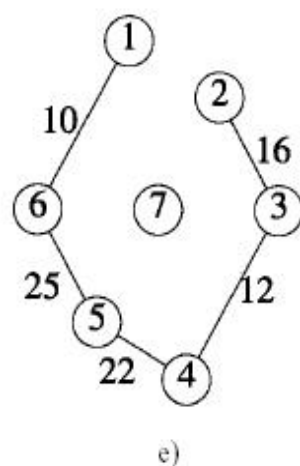
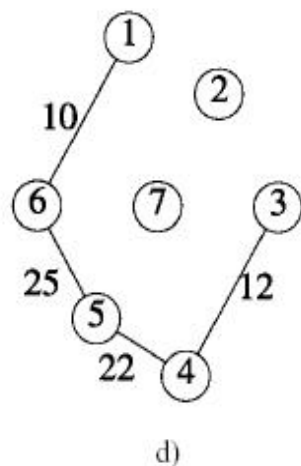
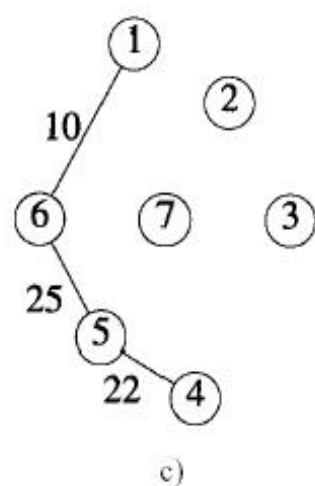
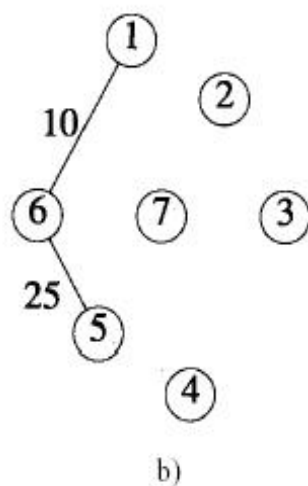
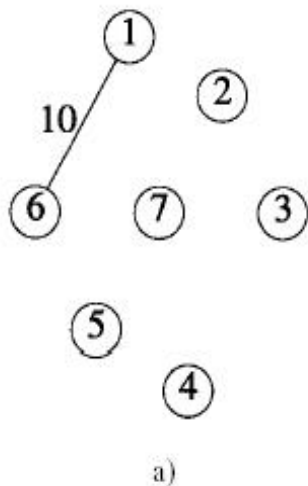
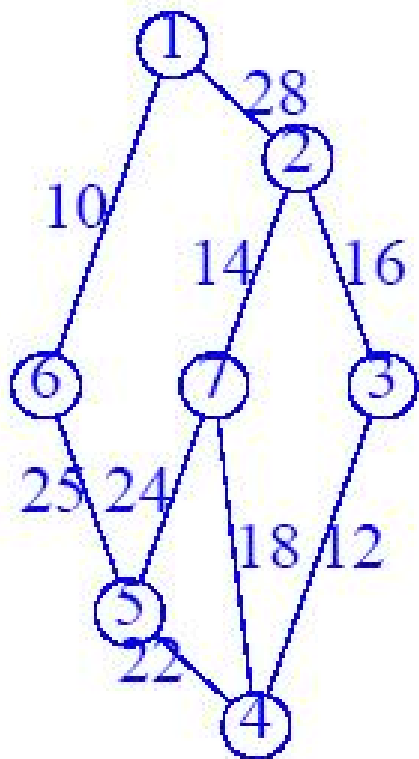
Prim's算法-伪代码

- ❑ 算法实现时将V-A中的节点按key值做成一个优先级队列(堆)Q。
- ❑ 每次从V-A中抽取距离最小的节点u, 并修改Q中与u相邻的节点的key值(decrease key)。
- ❑ 算法的伪代码如下:

```
 $Q \leftarrow V$   
 $key[v] \leftarrow \infty$  for all  $v \in V$   
 $key[s] \leftarrow 0$  for some arbitrary  $s \in V$   
while  $Q \neq \emptyset$   
  do  $u \leftarrow \text{EXTRACT-MIN}(Q)$   
    for each  $v \in \text{Adj}[u]$   
      do if  $v \in Q$  and  $w(u, v) < key[v]$   
        then  $key[v] \leftarrow w(u, v)$      $\triangleright$  DECREASE-KEY  
           $\pi[v] \leftarrow u$ 
```

At the end, $\{(v, \pi[v])\}$ forms the MST.

Prim's算法-实例



Prim's 算法-复杂度分析

$\Theta(V)$ total { $Q \leftarrow V$
 $key[v] \leftarrow \infty$ for all $v \in V$
 $key[s] \leftarrow 0$ for some arbitrary $s \in V$
while $Q \neq \emptyset$
 do $u \leftarrow \text{EXTRACT-MIN}(Q)$
 for each $v \in \text{Adj}[u]$
 do if $v \in Q$ and $w(u, v) < key[v]$
 then $key[v] \leftarrow w(u, v)$
 $\pi[v] \leftarrow u$

$|V|$ times { $degree(u)$ times {

Handshaking Lemma $\Rightarrow \Theta(E)$ implicit DECREASE-KEY's.

$$\text{Time} = \Theta(V) \cdot T_{\text{EXTRACT-MIN}} + \Theta(E) \cdot T_{\text{DECREASE-KEY}}$$

Prim's 算法-复杂度分析

$$\text{Time} = \Theta(V) \cdot T_{\text{EXTRACT-MIN}} + \Theta(E) \cdot T_{\text{DECREASE-KEY}}$$

Q	$T_{\text{EXTRACT-MIN}}$	$T_{\text{DECREASE-KEY}}$	Total
array	$O(V)$	$O(1)$	$O(V^2)$
binary heap	$O(\lg V)$	$O(\lg V)$	$O(E \lg V)$
Fibonacci heap	$O(\lg V)$ amortized	$O(1)$ amortized	$O(E + V \lg V)$ worst case

Prim's算法-正确性证明

对步数归纳

定理：对于任意 $k < n$, 存在一棵最小生成树包含算法前 k 步选择的边

归纳基础： $k=1$, 存在一棵最小生成树 T 包含边 $e=\{1,i\}$, 其中 $\{1,i\}$ 是所有关联 1 的边中权最小的.

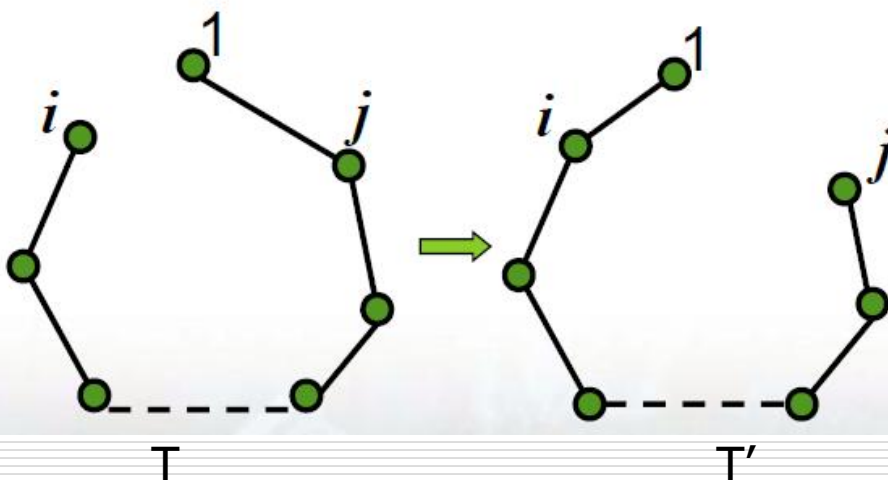
设 T 为一棵最小生成树, 假设 T 不包含 $\{1,i\}$, 则 $T \cup \{\{1,i\}\}$ 含有一条回路, 回路中关

联 1 的另一条边为 $\{1,j\}$,

令 $T' = (T - \{\{1,j\}\}) \cup \{\{1,i\}\}$,

则 T' 也是生成树,

且 $W(T') \leq W(T)$.



Prim's算法-正确性证明

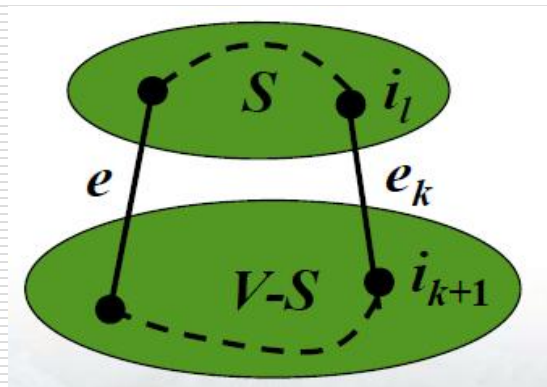
归纳步骤:

假设算法进行了 $k-1$ 步, 生成树的边为 $e_1, e_2, \dots, e_{k-1}$, 这些边的 k 个端点构成集合 S . 由归纳假设存在 G 的一棵最小生成树 T 包含这些边.

算法第 k 步选择了顶点 i_{k+1} , 则 i_{k+1} 到 S 中顶点的边权最小, 设这条边为 $e_k = \{i_{k+1}, i_l\}$. 假设 T 不含有 e_k , 则将 e_k 加到 T 中形成一条回路. 这条回路有另外一条连接 S 与 $V-S$ 中顶点的边 e , 令

$$T^* = (T - \{e\}) \cup \{e_k\},$$

则 T^* 是 G 的一棵生成树, 包含 $e_1, e_2, \dots, e_k$, $W(T^*) \leq W(T)$.

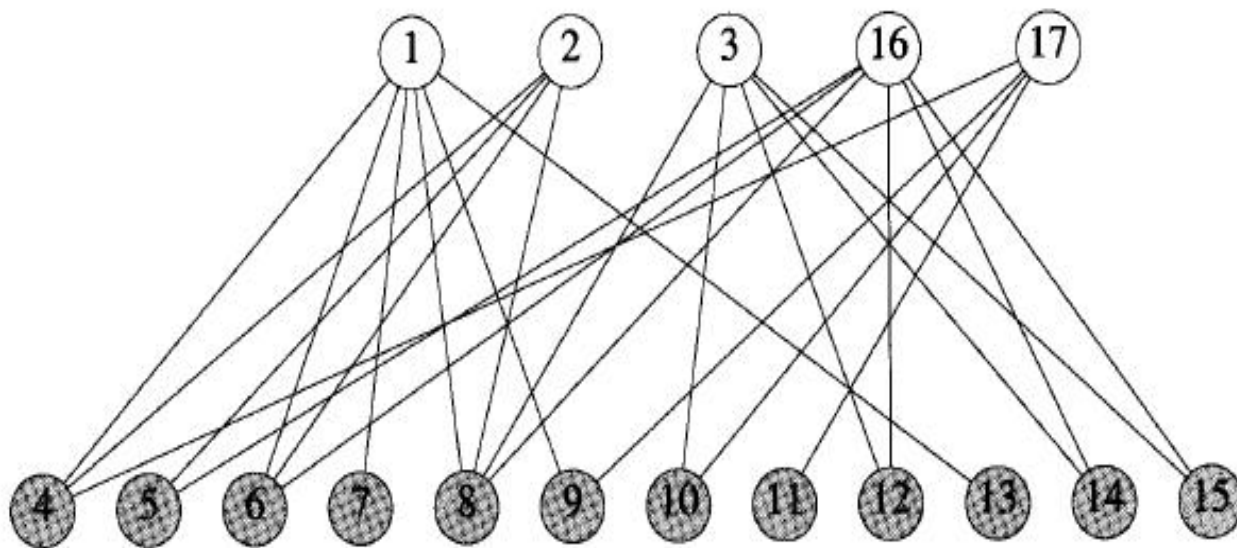


3.4.6 偶图覆盖问题

- 偶图是一个无向图，它的 n 个顶点分为集合 A 和集合 B ，且同一集合中的任意两个顶点无边相连。
 - A 的一个子集 A' 覆盖集合 B iff B 中每一顶点至少和 A' 中一顶点相连。覆盖 A' 的大小指 A' 中的顶点数目。
 - 在偶图中寻找最小覆盖的问题称为偶图覆盖(bipartite-cover)问题。
-

偶图覆盖问题-实例

- 例：具有17个顶点的二分图， $A=\{1, 2, 3, 16, 17\}$ 和 $B=\{4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15\}$,
- 子集 $A'=\{1, 16, 17\}$ 是 B 的最小覆盖：
- 1覆盖 $\{4, 6, 7, 8, 9, 13\}$; 16覆盖 $\{5, 6, 8, 12, 14, 15\}$; 17覆盖 $\{4, 9, 10, 11\}$ 。



集合覆盖问题

- 偶图覆盖等价于集合覆盖问题。集合覆盖问题是NP—难度问题。
 - 集合覆盖问题： n 个集合的族 $F = \{S_1, \dots, S_n\}$ 满足 $\bigcup_{i=1}^n S_i = U$, 设 $\{S_{i_1}, \dots, S_{i_k}\}$ 为 F 的子族 $\bigcup_{j=1}^k S_{i_j} = U$, 则称其为 U 的一个覆盖。
 - 集合覆盖问题是指找包含的集合数目最小的覆盖。
-

集合覆盖问题-实例

- 令 $F = \{S_1, \dots, S_5\}$, $U = \{4, 5, \dots, 15\}$, $S_1 = \{4, 6, 7, 8, 9, 13\}$, $S_2 = \{4, 5, 6, 8\}$, $S_3 = \{8, 10, 12, 14, 15\}$, $S_4 = \{5, 6, 8, 12, 14, 15\}$, $S_5 = \{4, 9, 10, 11\}$.
- $S' = \{S_1, S_4, S_5\}$ 是一个大小为3的覆盖, 没有更小的覆盖, S' 即为最小覆盖。
- 这个集合覆盖问题可变换为如上偶图, 即用顶点1, 2, 3, 16和17分别表示集合 S_1, S_2, S_3, S_4 和 S_5 , 顶点 j 表示集合 U 中的元素 j , $4 \leq j \leq 15$.
- 两个问题等价, 都是NP-难度问题

偶图覆盖问题-贪心算法设计

□ 贪心策略：选择覆盖**B**中那些尚未被覆盖的顶点数最多的**A**的节点

□ 算法伪代码：

```
m=0; //当前覆盖的大小
对于A中的所有i, New[i]=Degree[i]
对于B中的所有i, Cov[i]=false
while (对于A中的某些i, New[i]>0) {
    设v是具有最大的 New[i]的顶点;
    C[m++]=v;
    for (所有邻接于v的顶点j) {
        if (!Cov[j]) {
            Cov[j]= true;
            对于所有邻接于j的顶点, 使其 New[k]减1
        }
    }
}
if (有些顶点未被覆盖) 失败
else 找到一个覆盖
```

偶图覆盖问题-算法实现问题

- 设 $i \notin A'$, New_i 为 i 覆盖的 B 中尚未被覆盖的顶点数。
- New_i 为动态变化的量。需设计数据结构对其进行维护。
- 要求：取 New_i 最大的顶点 i
相应修改余下顶点的 New 值

使用二维数组存储New的值

- 更新New的时间为 $O(e)$ ，其中 e 为二分图中边的数目。
 - 若使用邻接矩阵，则需要花 $O(n^2)$ 的时间来寻找图中的边
 - 若用邻接链表，则需要 $O(n+e)$ 的时间
 - 逐步选择顶点所需时间为 $O(\text{Sizeof}(A))$ ，其中 $\text{Sizeof}(A)=|A|$ 。
 - A的所有顶点都有可能被选择，因此，所需步骤数为 $O(\text{Sizeof}(A^2))$
 - 覆盖算法总的算法复杂性为 $O(\text{Sizeof}(A^2)+n^2)$ 或 $O(\text{Sizeof}(A^2)+n+e)$
-

使用最大堆存储New的值

- 使用有序数组、最大堆等可将每步选择顶点的复杂性将为 $\Theta(1)$. 但利用有序数组, 在每步的最后需要对 New_i 值进行排序, 而排序所需要的时间通常比 $\text{Sizeof}(A)$ 大得多, 因此有序数组并不总能提高性能。
 - 如果利用最大堆, 则每一步都需要重建堆来记录New值的变化, 可以在每次New值减1时进行重建。这种减法操作可引起被减的New值最多在堆中向下移一层, 因此这种重建对于每次New值减1需 $\Theta(1)$ 的时间, 总共的减操作数目为 $O(e)$ 。因此在算法的所有步骤中, 维持最大堆仅需 $O(e)$ 的时间,
 - 因而利用最大堆时覆盖算法的总复杂性为 $O(n^2)$ 或 $O(n+e)$ 。
-

补充：连续背包问题

- 考虑 $0 \leq x_i \leq 1$ 而不是 $x_i \in \{0, 1\}$ 的连续背包问题。一种可行的贪心策略是：按价值密度非递减的顺序检查物品，若剩余容量能容下正在考察的物品，将其装入；否则，往背包中装入此物品的一部分。
 - 1) 对于 $n=3$, $w=[100, 10, 10]$, $p=[20, 15, 15]$ 及 $c=105$, 上述装入方法获得的结果是什么？
 - 2) 证明这种贪心算法总能获得最优解。
-

算法正确性证明

- 证明这种贪心算法总能产生最优解

设 $x=(x_1,...,x_n)$ 为贪心法产生的解;形式为 $(1,1,..., x_j,0...0)$,
其中 $0<x_j<1$; $y=(y_1,...,y_n)$ 是最优解;

- 设 k 是 $x_i \neq y_i$ 的最小下标, 则 $k \leq j$ 且 $y_k < x_k$
- 将 y_k 增加到 x_k , 并从 $\sum_{k < i \leq n} y_i w_i$ 减去 $(x_k - y_k)w_k$ (无论什么方式), 得到 $(z_k, z_{k+1}, \dots, z_n)$
- 证明 $\sum_{k \leq i \leq n} z_i p_i \geq \sum_{k \leq i \leq n} y_i p_i$

将 $(z_i - y_i)p_i$ 改写成 $(z_i - y_i)w_i p_i / w_i$

利用 $p_k / w_k \geq p_i / w_i$ for $i > k$ 可得到上述不等式。
